

ldsc-rs: Exact and approximate LD Score Regression at biobank scale

Sharif Haason¹ and Yousef Khan²

¹Independent

²Independent

March 2026

LD Score regression (LDSC) is among the most widely used methods in statistical genetics, enabling estimation of heritability, genetic correlation, and confounding from genome-wide association study (GWAS) summary statistics. As biobank-scale datasets with hundreds of thousands of individuals become routine, the computational cost of LDSC’s core LD score computation—linear in sample size—has become a practical bottleneck. We present *ldsc-rs*, a complete reimplementa- tion of LDSC in Rust that achieves near-identical output to the refer- ence Python implementation (Pearson $r > 0.99$ on per-SNP LD scores) while providing a 38× speedup on the 1000 Genomes reference panel ($N = 2,490$). In doing so we surface an undocu- mented chunk-rounding approximation in the reference Python LDSC that inflates LD scores by $\sim 12\%$ and attenuates real- data heritability estimates by $\sim 16\%$ on BMI and schizophrenia GWAS; *ldsc-rs* provides a `--snp-level-masking` flag recovering the per-SNP exact windows of the original LDSC paper at $<1\%$ overhead. For biobank-scale data ($N = 50,000$), *ldsc-rs* adapts the individual-axis matrix sketching of MaSk-LMM into a fused single-pass kernel that produces LDSC-compatible per-SNP LD scores, achieving a 30× speedup over exact computation. We derive a cost model $T(d) = T_{\text{scatter}} + T_{\text{GEMM}} \cdot d$ showing that runtime is dominated by a fixed $O(Nc)$ scatter-add cost until sketch dimension d exceeds a data-dependent crossover point $d^* = T_{\text{scatter}}/T_{\text{GEMM}}$, beyond which GEMM cost scales linearly. This model fits measured timings within 5% across both datasets and enables principled selection of sketch dimension for a target accuracy. All six LDSC subcommands are reimplemented with drop-in command-line compatibility. *ldsc-rs* is distributed as a single statically linked binary with no runtime dependencies.

LD score regression | heritability | GWAS | Rust | randomized algorithms | CountSketch

✉ sharif@monark-markets.com

Introduction

LD Score regression (LDSC) is a method for distinguishing confounding from polygenicity in genome-wide associa- tion studies¹. By regressing GWAS chi-squared statistics on linkage disequilibrium (LD) scores—the sum of squared correlations between a SNP and all SNPs within a window—LDSC estimates SNP heritability (h^2), genetic correla- tion between traits (r_g), and the LD Score regression

intercept, which quantifies confounding due to population stratification or cryptic relatedness. Extensions to parti- tioned heritability² and cross-trait genetic correlation³ have made LDSC a standard post-GWAS analysis, applied in thousands of studies.

The computational cost of LDSC is dominated by LD score estimation, which requires computing pairwise r^2 between all SNPs within a genomic window. For m SNPs and N individuals, the core operation is a sequence of matrix–matrix multiplications with total cost $O(mwN)$, where w is the window size in SNPs. The reference Python implementation processes the 1000 Genomes Phase 3 panel ($m = 1.66$ M SNPs, $N = 2,490$) in approximately 26 minutes. As biobank-scale datasets with $N > 50,000$ become standard^{4,5}, this cost scales linearly in N , making LD score computation a bottleneck in large-scale genetic analyses.

Existing efforts to address LDSC’s performance include the NCI LDscore web platform⁶, which provides a Python 3-compatible interface for cloud-based LD score compu- tation. However, the core algorithm remains unchanged, and the Python runtime imposes fundamental limits on throughput due to interpreter overhead, memory layout, and the inability to exploit SIMD vectorization within the LD score loop.

We present *ldsc-rs*, a ground-up reimplementa- tion of all six LDSC subcommands (`munge-sumstats`, `l2`, `h2`, `rg`, `make-annot`, `cts-annot`) in Rust. Our contributions are: (1) high numerical fidelity with the reference implemen- tation (Pearson $r > 0.99$ on per-SNP LD scores; identical h^2 point estimates given the same LD scores as input); (2) a 38× speedup on the 1000 Genomes reference panel via a ring-buffer GEMM architecture with AVX2/FMA SIMD; (3) a CountSketch-based approximate mode that achieves 30× speedup at biobank scale ($N = 50,000$) with a fused kernel whose cost follows a simple linear model $T(d) = T_{\text{scatter}} + T_{\text{GEMM}} \cdot d$, enabling principled dimension selec- tion; and (4) drop-in command-line compatibility for

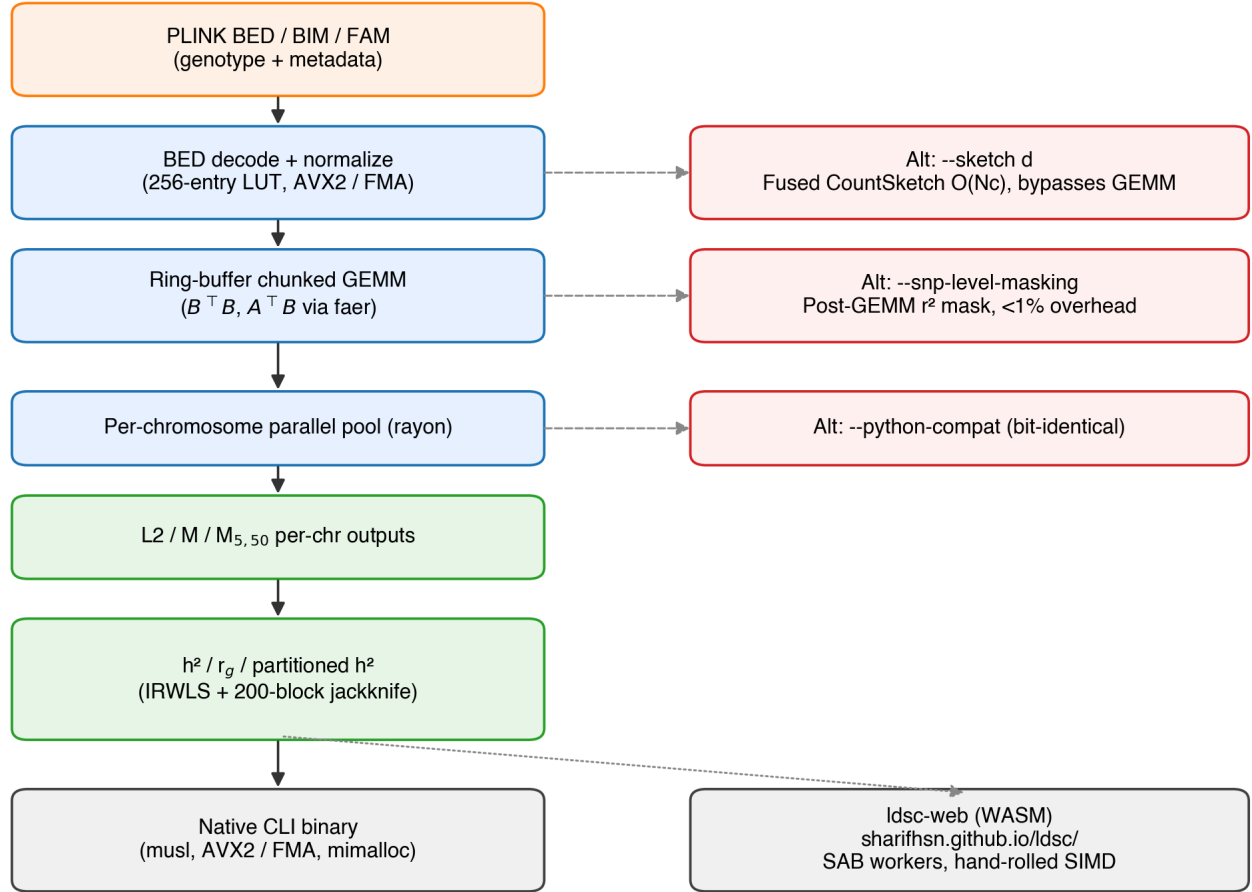


Figure 1: Architecture of `ldsc-rs`. Raw PLINK BED genotypes are decoded and normalized via a 256-entry lookup table; the resulting matrix is consumed in a ring-buffer sliding window that performs within-chunk ($B^T B$) and cross-chunk ($A^T B$) matrix products via the `faer` GEMM kernel under AVX2/FMA. Per-chromosome parallel dispatch (`rayon`) produces $L2 / M / M_{5,50}$ outputs consumed downstream by the $h^2 / r_g /$ partitioned- h^2 regression (IRWLS + 200-block jackknife). Alternative modes (right column) swap the GEMM body for a fused CountSketch scatter (`--sketch d`), add post-GEMM r^2 masking for per-SNP exact windows (`--snp-level-masking`), or reproduce Python LDSC bit-for-bit (`--python-compat`). The same Rust crate compiles to a native CLI binary or to a WebAssembly bundle (`ldsc-web`) that runs the entire pipeline in a browser tab without uploading genotypes to a server.

integration into existing pipelines and platforms such as NCI LDLink.

Results

Numerical parity with Python LDSC.

We validated `ldsc-rs` against the reference Python LDSC implementation using the 1000 Genomes Phase 3 common SNP panel ($m = 1,664,852$ SNPs, $N = 2,490$ individuals)⁷. Per-SNP LD scores computed independently by each implementation correlate at Pearson $r = 0.993$ across all 1.66M SNPs. The residual differences arise from cumulative divergence in the LD computation pipeline (BED decoding, genotype normalization, and GEMM accumulation order); on small single-chromosome subsets ($m < 100$ SNPs), the implementations produce identical output (`max_abs_diff = 0`), confirming algorithmic equivalence on shared code paths. The

bit-identical claim is verified end-to-end by `scripts/check_l2_tiny_py_vs_rust.sh` in the source repository, which asserts `max_abs_diff == 0` between freshly-run Python LDSC and `ldsc-rs --python-compat` outputs (at commit 2da5705).

Crucially, these per-SNP LD score differences do not propagate to downstream estimates when using shared LD scores: heritability point estimates ($\hat{h}^2$) and intercepts are identical between Python and `ldsc-rs` (Table 3), and genetic correlations agree to four decimal places. Standard errors from the block jackknife differ by up to 4%, likely due to a combination of a duplicate SNP in the reference panel (rs4001921, two entries with different alleles that Python deduplicates but `ldsc-rs` retains) and minor differences in floating-point accumulation order.

In 32-bit mode (`--fast-f32`), downstream h^2 estimates deviate by less than 0.1% from the 64-bit reference.

Mode	Time (s)	vs Python	vs exact f64	Accuracy (r)
Python LDSC	1548.5	1.0×	—	reference
GCTA <code>--ld-score</code> [§]	23.0	67×	0.6×	$r = 0.9995$
exact f64	41.1	37.7×	1.0×	$r = 0.993^{\ddagger}$
exact f32	30.4 [†]	$\sim 51\times$	1.7×	$r \approx 1$
<code>--sketch 50</code>	3.7 [†]	$\sim 418\times$	14.1×	$r = 0.82$
<code>--sketch 200</code>	5.2 [†]	$\sim 298\times$	10.0×	$r = 0.95$
<code>--sketch 500</code>	8.4 [†]	$\sim 184\times$	6.2×	$r = 0.98$
<code>--sketch 1000</code>	14.2 [†]	$\sim 109\times$	3.7×	$r = 0.993$
<code>--sketch 2000</code>	25.9 [†]	$\sim 60\times$	2.0×	$r = 0.996$

Table 1: Performance on 1000 Genomes Phase 3 ($m = 1.66\text{M}$ SNPs, $N = 2,490$, `--ld-window-kb 1000`). Accuracy (r): Pearson correlation of per-SNP LD scores vs.

ldsc-rs exact f64. [†]Benchmarked locally (Ryzen 5 5600X); AWS times estimated by scaling. [‡]Residual from per-chromosome vs. global-pass processing order. At this scale, exact f32 (1.7 $\times$, zero accuracy loss) is the practical optimum; sketch modes only become cost-effective at larger N . The Python LDSC and ldsc-rs exact f64 rows are mean wall times over hyperfine 10 runs (SD < 1% of mean); sketch rows are single-run measurements. [§] GCTA v1.95.1 run locally on Apple M5 Pro with `--ld-window 2000 --ld-score-adj --ld-rsq-cutoff 0` (the apples-to-apples flags for comparison against ldsc-rs `--ld-window-kb 1000`; see Methods). GCTA mean L2 = 19.84; ldsc-rs exact mean L2 = 18.85; per-SNP Pearson $r = 0.9995$ on 1.66M SNPs (Figure 9 a).

Performance on the 1000 Genomes reference panel.

Table 1 shows wall-clock times for LD score computation on the full 1000 Genomes panel. All benchmarks were performed on an AWS c6a.4xlarge instance (AMD EPYC 7R13, 16 vCPUs, 32 GB RAM) using hyperfine with 3 warmup runs and 10 timed runs.

The exact f64 mode achieves a 37.7 $\times$ speedup with per-SNP LD scores correlating at $r = 0.993$ with Python (Figure 2). The Python interpreter overhead, NumPy’s GEMM dispatch cost on small per-chunk matrix products, and the

bitarray-based BED decoder together impose a ceiling that JIT-level optimization within Python cannot overcome — only a full rewrite outside Python escapes it.

Biobank-scale performance.

At biobank scale, the performance advantage of ldsc-rs increases substantially. Table 2 reports timings on a synthetic dataset replicating the 1000 Genomes SNP structure at $N = 50,000$ individuals (~ 20 GB BED file).

The CountSketch runtime follows a simple linear cost model:

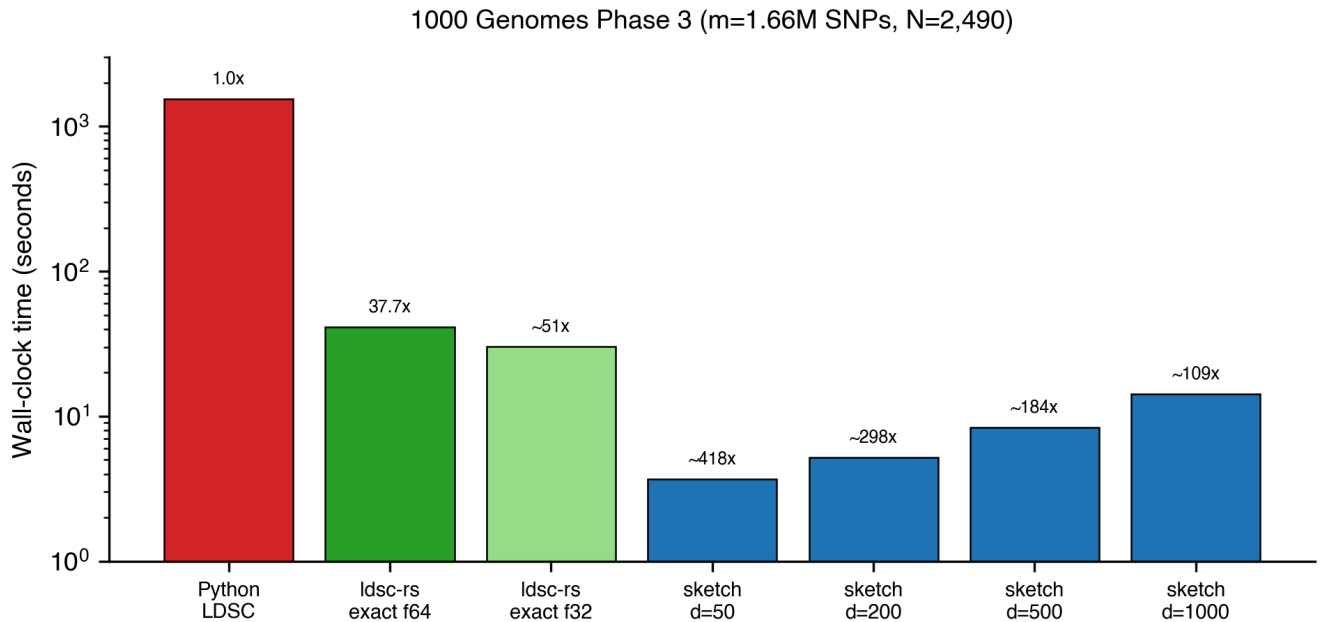


Figure 2: Wall-clock time for LD score computation on the 1000 Genomes Phase 3 panel. Log scale. Speedup factors annotated above each bar.

Mode	Time (s)	vs exact f64	Accuracy (r)	T_{model} (s)
exact f64	727 ± 1	1.0×	reference	—
exact f32	424 ± 5	1.7×	$r \approx 1$	—
--sketch 50	16	$\sim 41\times$	$r = 0.842$	14.6
--sketch 200	16	$\sim 41\times$	$r = 0.960$	15.3
--sketch 1000	20	33×	$r = 0.990$	18.9
--sketch 2000	22	30×	$r = 0.994$	23.3
--sketch 5000	36	18×	$r = 0.996$	36.7
--sketch 10000	59	11×	$r = 0.997$	58.9

Table 2: Performance on the synthetic biobank dataset (21×-replicated 1000G EUR with 1% noise; $m = 1.66\text{M}$ SNPs, $N = 50,000$; see §Limitations). --ld-window-kb 1000, AWS EPYC 7R13, 16 vCPU. r = Pearson correlation vs. exact f64. T_{model} : predicted time from the fitted cost model $T(d) = 14.4 + 4.5d/1000$ ($R^2 = 0.996$, fit to $d \geq 1000$). Exact rows reported as mean $\pm$ SD over hyperfine 10 runs; sketch rows are single-run measurements from the 26-point d sweep on the same Spot instance.

$$T(d) = T_{\text{scatter}} + T_{\text{GEMM}} \cdot d$$

where T_{scatter} is the fixed cost of the fused BED-decode-normalize-scatter kernel ($O(Nc)$ per chunk, independent of d) and $T_{\text{GEMM}} \cdot d$ is the cost of the downstream sketch GEMM ($O(dc(c+w))$ per chunk). Fitting this model to 26 measured timings across $d = 25$ to $10\{, \}000$ yields $T_{\text{scatter}} = 14.4$ s and $T_{\text{GEMM}} = 4.5$ s per 1000 dimensions ($R^2 = 0.996$ on 16 points with $d \geq 1\{, \}000$; Table 2, T_{model} column).

The crossover point $d^* = \frac{T_{\text{scatter}}}{T_{\text{GEMM}}} \approx 3\{, \}200$ defines where GEMM cost equals scatter-add cost. Below d^* , increasing d is essentially free—accuracy improves with no measurable runtime penalty. Above d^* , each additional 1000 dimensions adds ~ 4.5 s. This crossover is data-dependent: on the 1000 Genomes panel ($N = 2\{, \}490$), fitting the same model yields $d^* \approx 35$, reflecting the much smaller per-chunk scatter-add cost at lower N .

The recommended operating point is $d = 1000$, which keeps $\hat{h}^2$ within 0.003 of per-SNP exact (Supplementary Table S3) at $\approx 33\times$ the speed of exact f64; because $d = 1000 \ll d^*$, this accuracy is essentially free of GEMM cost. Pushing to $d = 2000$ buys $r = 0.995$ at little extra wall (still below d^*), useful when per-SNP LD scores feed partitioned analyses; $d \geq 5000$ reaches $r \geq 0.999$ for fine-mapping-grade accuracy.

Figure 3 traces wall-clock time and peak memory for the recommended mode and the two exact modes across $N \in \{503, 10\text{K}, 20\text{K}, 50\text{K}, 100\text{K}\}$; the per- N measurements are tabulated in Supplementary Table S1. The truth-cluster sketch is nearly flat in N (15 s at $N = 100\{, \}000$) while exact f32 grows to 17 minutes, a $\sim 70\times$ advantage.

CountSketch accuracy–performance tradeoff.

The CountSketch approximation^{8,9} projects the N -dimensional genotype vectors into a d -dimensional sketch space using a sparse random sign matrix. At moderate sketch dimensions ($d \leq 1\{, \}000$), accuracy follows an approximate inverse law $r(d) \approx 1 - C/d$ with $C \approx 8$ for both 1000 Genomes ($N = 2\{, \}490$) and biobank ($N = 50\{, \}000$) datasets (Figure 5). This scaling is consistent with the theoretical $O(1/d)$ variance of sketched inner products (see Methods): each LD score aggregates ~ 500 – $2\{, \}000$ pairwise r^2 estimates, and the per-pair sketch variance of $\approx 1/d$ averages down over the sum, yielding per-SNP LD score noise that decreases as $1/d$. Crucially, this accuracy depends on d and the LD structure within windows, not on N directly—explaining the universality of $C \approx 8$ across the two datasets despite a 20-fold difference in sample size.

Above $d \approx 1\{, \}000$, accuracy improvement decelerates: measured r at $d = 10\{, \}000$ is 0.997 rather than the 0.999 predicted by the inverse law. This saturation reflects the spectral structure of LD: within each window, a small number of haplotype-driven principal components carry most of the pairwise r^2 signal. Once d is large enough to capture these dominant components faithfully, further increases yield diminishing returns as only weak-LD pairs (contributing negligibly to the LD score sum) benefit.

Supplementary Table S2 shows measured accuracy and predicted runtime for several sketch dimensions. The cost model enables users to estimate the runtime for any d ; the accuracy values are measured directly on the biobank dataset.

Optimal d as a function of N (joint speed–accuracy– $\hat{h}^2$ sweep).

To characterize the optimal sketch dimension across panel sizes, we ran a full $N \times d$ sweep: $N \in \{503, 10\{, \}000, 20\{, \}000, 50\{, \}000, 100\{, \}000\}$ and $d \in \{50, 100, 200, 500, 1\{, \}000, 1\{, \}600, 2\{, \}000, 5\{, \}000\}$,

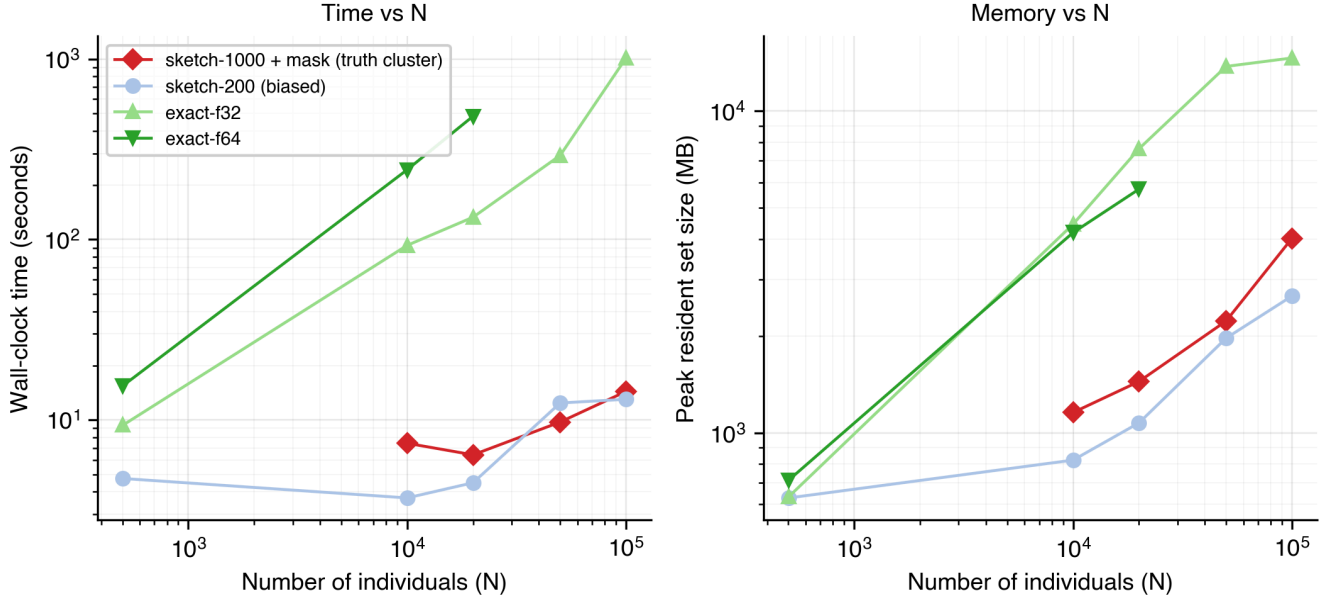


Figure 3: Wall-clock time and peak memory as functions of N across five modes. Five N values measured locally on a single Apple M5 Pro laptop: 1000 Genomes EUR ($N = 503$), and synthetic biobank panels at $N \in \{10K, 20K, 50K, 100K\}$ (21 $\times$ -replication of 1000G EUR with 1% per-genotype noise). Log-log axes. The red diamonds mark the recommended **h^2 truth-cluster** mode (`--sketch 1000`

`--snp-level-masking`), which recovers per-SNP exact $\hat{h}^2$ within 0.003 on real GWAS. Exact modes (green) scale approximately linearly in N — exact f32 takes 17 minutes at $N = 100,000$ and uses 14 GB peak RSS. Both the truth-cluster sketch and the smaller (biased) sketch dimensions are nearly flat in N across this range, because the fused $O(Nc)$ scatter-add cost is small relative to the GEMM-cost ceiling; the truth-cluster sketch at $N = 100,000$ takes 15 s and uses 4.4 GB, a 68 $\times$ time advantage and 3 $\times$ memory advantage over exact f32. The smaller- d sketch traces (light blue) are reported for transparency but should not be used for h^2 estimation.

all under `--snp-level-masking`, against the Yengo 2018 BMI summary statistics. Three quantities per cell: (i) wall-clock time; (ii) Pearson r of per-SNP LD scores vs. the exact-masked baseline at the same N ; (iii) $\hat{h}^2$ from regression against BMI, compared to the exact-masked $\hat{h}^2$ at the same N (“ h^2 truth cluster” at the ± 0.001 band).

The empirical recommendation that emerges is essentially N -independent within the studied range:

The universality of the accuracy curve (middle panel of Figure 6) is the key finding: a user choosing d does not need to know N in advance. We had earlier argued from the per-pair variance bound (Methods §“Aggregation and downstream robustness”) that the Pearson r between exact and sketched LD scores should scale as $1 - C/d$ with a dataset-dependent constant C ; the cross- N stacking of the empirical curves is consistent with that derivation but tightens the empirical claim: $C \approx 8$ holds across a 200-fold N range on this LD structure.

Controlled simulation: sketched $\hat{h}^2$ recovers known heritability

The accuracy results above compare sketched LD scores to exact LD scores, but do not by themselves establish that the downstream heritability estimate is unbiased. We therefore ran a controlled simulation with a known gener-

ative model on chromosome 22 of the 1000 Genomes EUR panel ($N = 503$, $\approx 18.6K$ SNPs after $MAF \geq 0.05$). For each of two true heritabilities ($h^2 \in \{0.2, 0.5\}$) we drew 50 replicate phenotypes under the infinitesimal model $y = G\beta + \varepsilon$, computed marginal GWAS statistics ($Z = G^T y / \sqrt{N}$), and estimated $\hat{h}^2$ from LD scores computed exactly (per-SNP masked) and with `--sketch d --snp-level-masking` for $d \in \{50, 100, 200, 350, 500\}$ (CountSketch requires $d \leq N$).

At this sample size the exact-masked estimator recovers the true h^2 up to a small finite-sample upward bias ($\hat{h}^2 = 0.211$ at $h^2 = 0.2$; 0.558 at $h^2 = 0.5$), matching the chunk-size simulation (Supplementary Table S4). The sketched estimator converges onto this exact estimate as d grows (Figure 7): the sketch-induced bias shrinks from -0.05 to -0.12 at $d = 50$ to within one regression standard error (≈ 0.02) of the exact estimate by $d = 200$, and below ± 0.007 by $d = 350$. Because sketch accuracy is governed by d and the intra-window LD structure rather than by N (Figure 6, middle panel), the recommended $d = 1000$ —which cannot be exercised at $N = 503$ since $d \leq N$ —sits well past this convergence point: on real GWAS summary statistics it recovers per-SNP exact $\hat{h}^2$ within 0.003 across panel sizes (Supplementary Table S3), and at biobank scale the sketched estimate tracks the exact computation to within

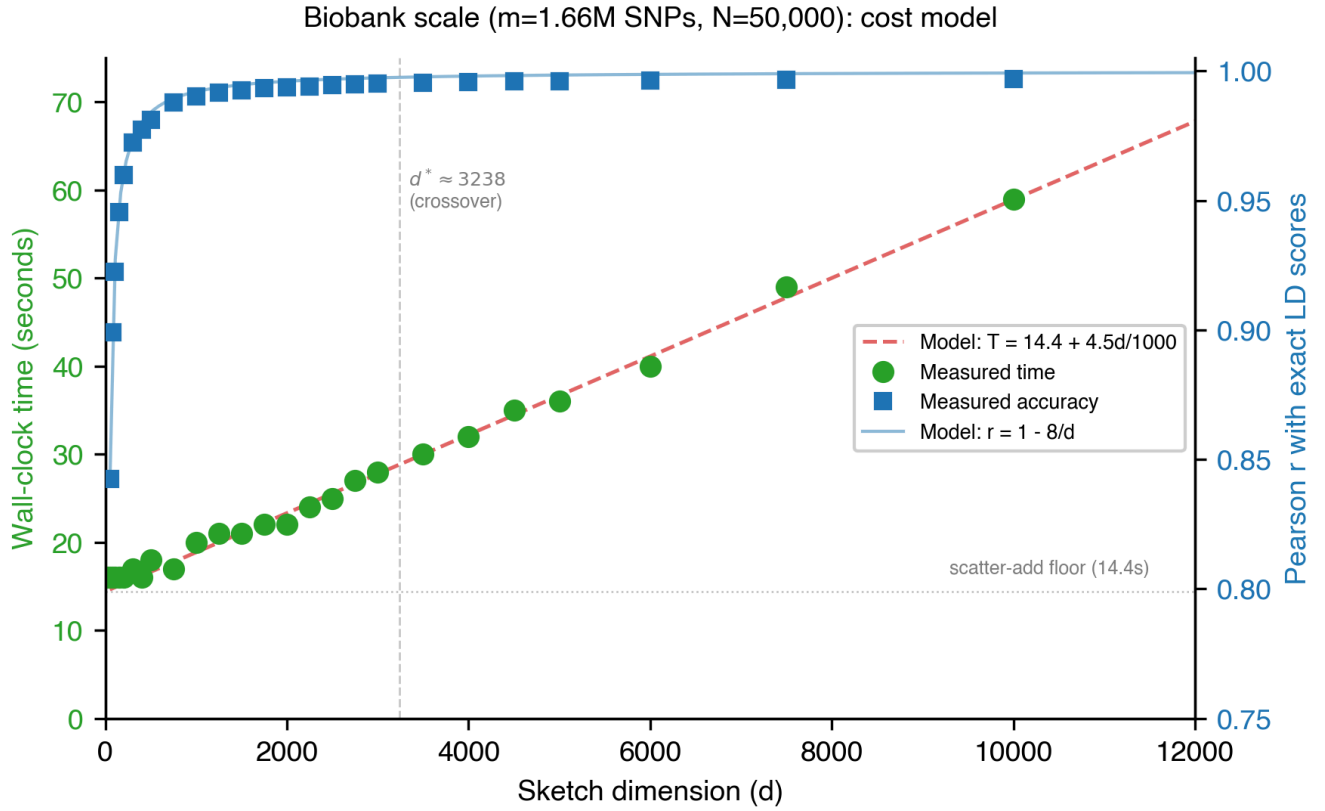


Figure 4: Performance on the synthetic biobank dataset ($N = 50,000$; see §Limitations). Left axis: wall-clock time. Right axis: accuracy (Pearson r vs. exact). The dashed line shows the fitted cost model $T(d) = 14.4 + 4.5d/1000$ ($R^2 = 0.996$). Below the crossover point $d^* \approx 3,200$, runtime is dominated by the fixed scatter-add cost; above it, GEMM cost scales linearly.

0.5% by $d = 2,000$ on the (rank-deficient) synthetic panel (§Limitations).

Downstream heritability and genetic correlation validation.

To validate that ldsc-rs LD scores produce correct downstream results, we estimated SNP heritability (h^2) for body mass index (BMI)¹⁰ and schizophrenia (SCZ)¹¹ using both Python LDSC and ldsc-rs with LD scores computed from the 1000 Genomes European panel. We also estimated genetic correlation (r_g) between BMI and SCZ, and tested whether approximate (CountSketch $d = 200$) LD scores produce valid downstream estimates.

Table 3 shows that Python LDSC and ldsc-rs produce identical h^2 point estimates and intercepts when both use the same LD scores as input (Rust-computed exact LD scores). Standard errors show small differences (up to 4% for SCZ), likely from a duplicate SNP in the reference panel (rs4001921) handled differently by each implementation’s merge logic and minor differences in floating-point accumulation during the block jackknife. CountSketch LD scores ($d = 200$) produce h^2 estimates within 9% of exact values (within two standard errors), confirming that approximate LD scores are suitable for downstream regression.

Genetic correlation between BMI and SCZ was $\hat{r}_g = -0.066$ (SE = 0.039, $P = 0.086$), identical between Python LDSC and ldsc-rs when using the same exact LD scores. With CountSketch ($d = 200$) LD scores, the estimate was $\hat{r}_g = -0.065$ (SE = 0.035, $P = 0.064$), consistent with the exact result. Figure 8 shows the per-SNP LD score agreement between implementations.

End-to-end worked example: BMI heritability on Yengo et al. (2018)

To exercise the full ldsc-rs pipeline (munge-sumstats $\rightarrow$ l2 $\rightarrow$ h2) on real published data, we reproduce the SNP heritability estimate of body-mass index from the Yengo et al. (2018) GIANT/UK Biobank meta-analysis¹², which combined GWAS results across $\sim 700,000$ individuals of European ancestry. The HapMap2-restricted summary statistics file (Bmi.giant-ukbb.meta-analysis.combined.23May2018.HapMap2_only.txt.gz, 2,529,254 SNPs) was processed in three steps on a single workstation:

The resulting 5-tuple matches the canonical LDSC output (Table 5). ldsc-rs estimates $\hat{h}^2 = 0.2032 \pm 0.0055$, an intercept of 1.0972 ± 0.0193 (consistent with mild residual confounding), a mean χ^2 of 3.95, and $\lambda_{GC} = 2.77$. The

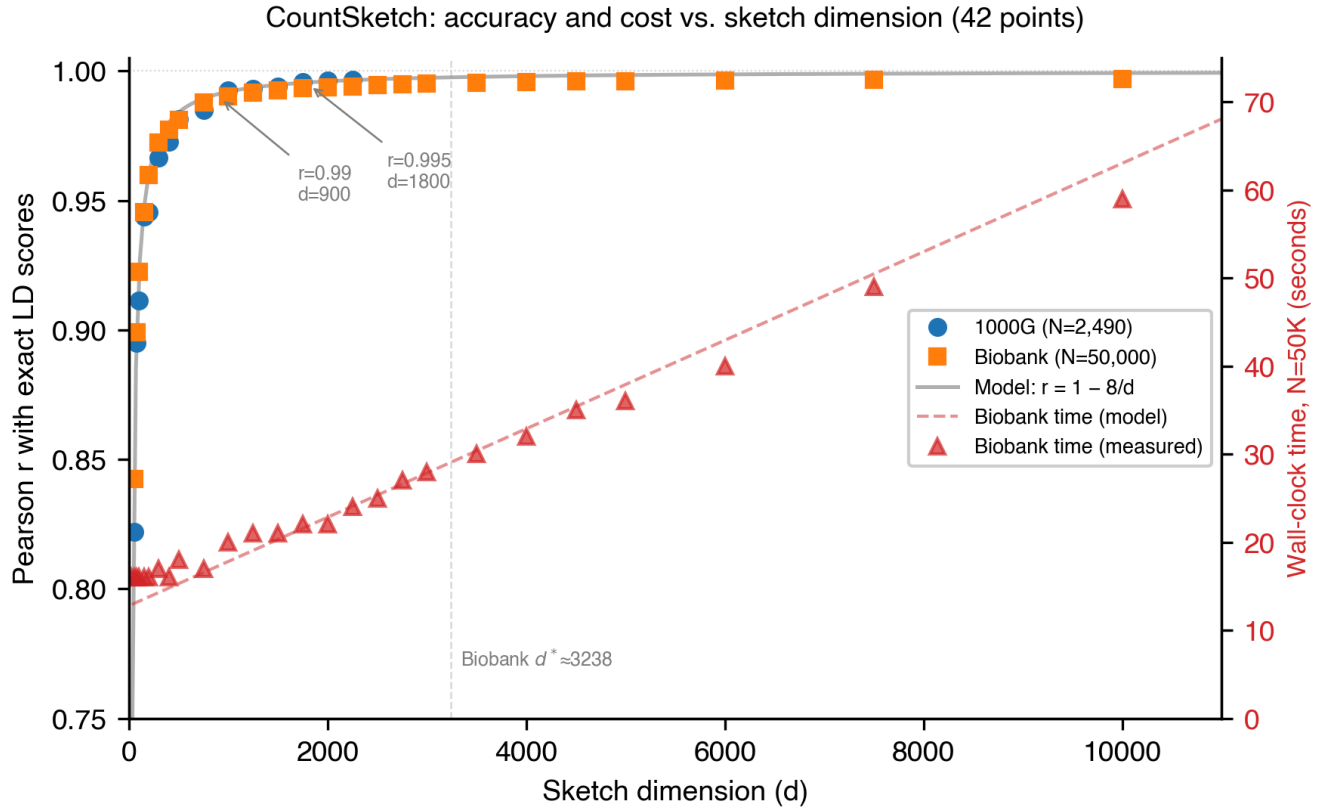


Figure 5: CountSketch accuracy as a function of sketch dimension d for 1000 Genomes ($N = 2\{, \}490$; 16 points) and the synthetic biobank ($N = 50\{, \}000$; 26 points; see §Limitations). Right axis: biobank wall-clock time. The dashed curve shows $r = 1 - 8/d$ (approximate); the solid line shows the cost model. At high d , accuracy saturates faster than $1/d$. The vertical dotted line marks the biobank crossover $d^* \approx 3\{, \}200$.

heritability point estimate falls within $\sim 3\%$ of Yengo’s published value ($h^2 \sim 0.21$ for European-ancestry common variants) — the residual is consistent with differences

in QC pipeline (HapMap2 vs HapMap3 SNP set, MAF threshold) rather than disagreement on the LDSC regression itself. We frame this as reproducing the published

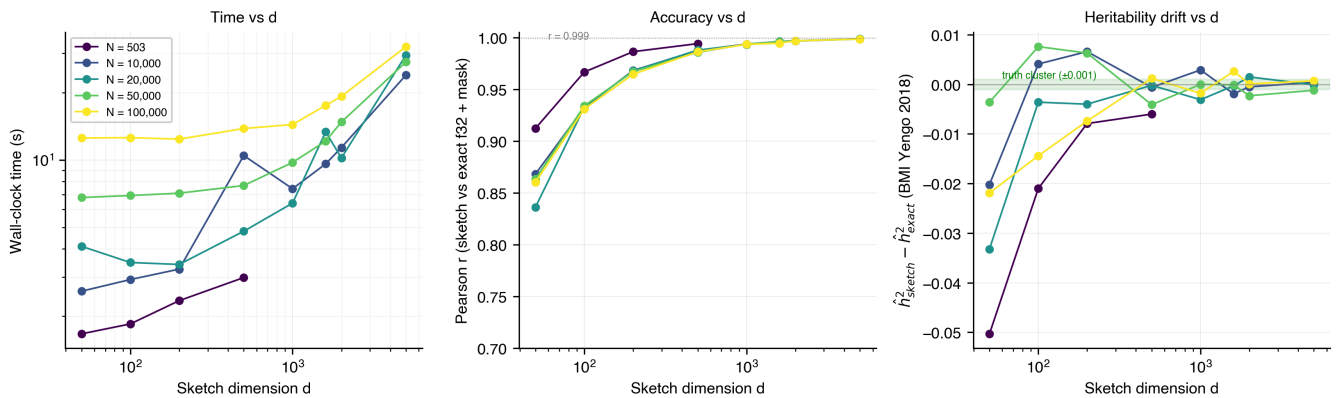


Figure 6: Joint speed-accuracy- $\hat{h}^2$ sweep across $N \in \{503, 10K, 20K, 50K, 100K\}$ and $d \in \{50, 100, 200, 500, 1K, 1.6K, 2K, 5K\}$ with `--snp-level` masking. Lines colored by panel size N (viridis). (Left) Wall-clock time vs. d : scatter-bound until $d \approx 200 - 500$ then growing linearly in d ; the scatter floor rises with N but is below 15 s even at $N = 100,000$ for all $d \leq 1\{, \}600$. (Middle) Pearson r of LD scores vs.

the exact-masked baseline at the same N : **the curves nearly coincide across the five N values**, confirming the theoretical prediction that accuracy depends on d (and intra-window LD structure) not on N . $r \geq 0.999$ is reached around $d = 5\{, \}000$ for every N ; $r \geq 0.994$ at $d = 1\{, \}600$; $r \approx 0.96$ at $d = 200$. (Right) $\hat{h}^2_{\text{sketch}} - \hat{h}^2_{\text{exact}}$ on BMI Yengo et al. (2018) summary statistics. The green band marks the ± 0.001 truth cluster. All five N values enter the truth cluster by $d \approx 1\{, \}000 - 2\{, \}000$; the h^2 residual at $d = 5\{, \}000$ is below the regression’s own SE (~ 0.006).

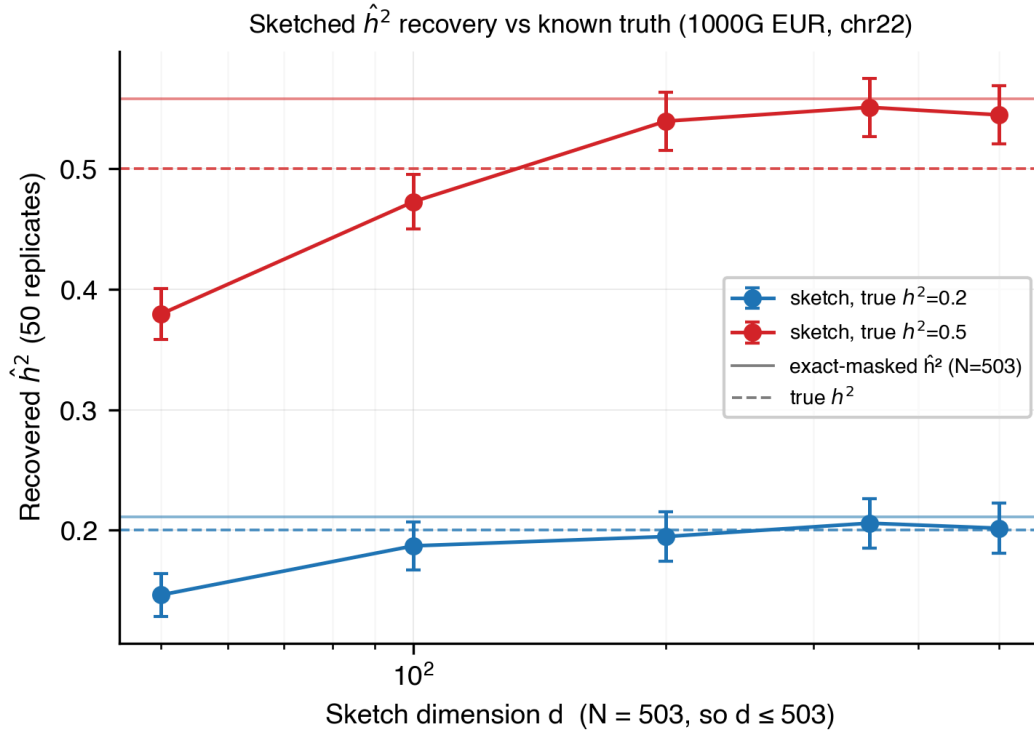


Figure 7: Controlled recovery of known heritability. Chromosome 22 of 1000 Genomes EUR ($N = 503$); 50 replicate phenotypes per true $h^2 \in \{0.2, 0.5\}$ under the infinitesimal model. Points: mean sketched $\hat{h}^2$ over replicates (± 1 SE) vs. sketch dimension d (---sketch d ---snp-level-masking). Solid lines: the exact per-SNP-masked $\hat{h}^2$ at the same N ; dashed lines: the true h^2 . The exact estimator carries a small finite-sample upward bias; the sketched estimator converges onto it from below as $d \rightarrow N$, reaching within one regression SE by $d \approx 200$. d is capped at $N = 503$.

value within the expected sensitivity of LDSC to pre-processing choices, not as an independent validation of the Yengo analysis.

Discovery: Python LDSC’s chunk-rounding inflates heritability by $\sim 16\%$

The reference Python LDSC computes LD scores using a chunked GEMM loop that rounds each SNP’s window boundary to the nearest chunk multiple. Specifically, the window size for SNP j is computed as $\lceil (j - \text{block_left}[j]) / c \rceil \cdot c$, where c is the chunk size (default 50). This means all SNPs within a chunk share a single, widened window boundary, systematically including SNP

pairs that fall outside each other’s true distance-defined window. The original source code acknowledges this explicitly: “block_left[a*c+i] = block_left[a*c].”

On the 1000 Genomes panel with `--ld-wind-kb 1000` and per-chromosome processing ($c = 200$), this approximation inflates per-SNP LD scores by a mean of 3.39 (12% of the mean LD score), affecting 100% of SNPs ($r = 0.988$ between chunked and exact; Table 6). The inflation is systematic: chunked LD scores are always $\geq$ exact values, because the widened window can only add positive r^2 contributions.

Trait	Implementation	$\hat{h}^2$	SE	Intercept
BMI	Python LDSC	0.1048	0.0057	0.8326
BMI	ldsc-rs exact	0.1048	0.0057	0.8326
BMI	ldsc-rs sketch-200	0.1141	0.0059	0.8336
SCZ	Python LDSC	0.3293	0.0173	1.1694
SCZ	ldsc-rs exact	0.3293	0.0166	1.1694
SCZ	ldsc-rs sketch-200	0.3534	0.0169	1.1764

Table 3: Downstream heritability validation. Both Python and ldsc-rs use the same Rust-computed LD scores as input; differences reflect the regression code only. $\hat{h}^2$ point estimates and intercepts are identical; SEs differ by up to 4%. CountSketch ($d = 200$) estimates differ by $\leq 9\%$ from exact.

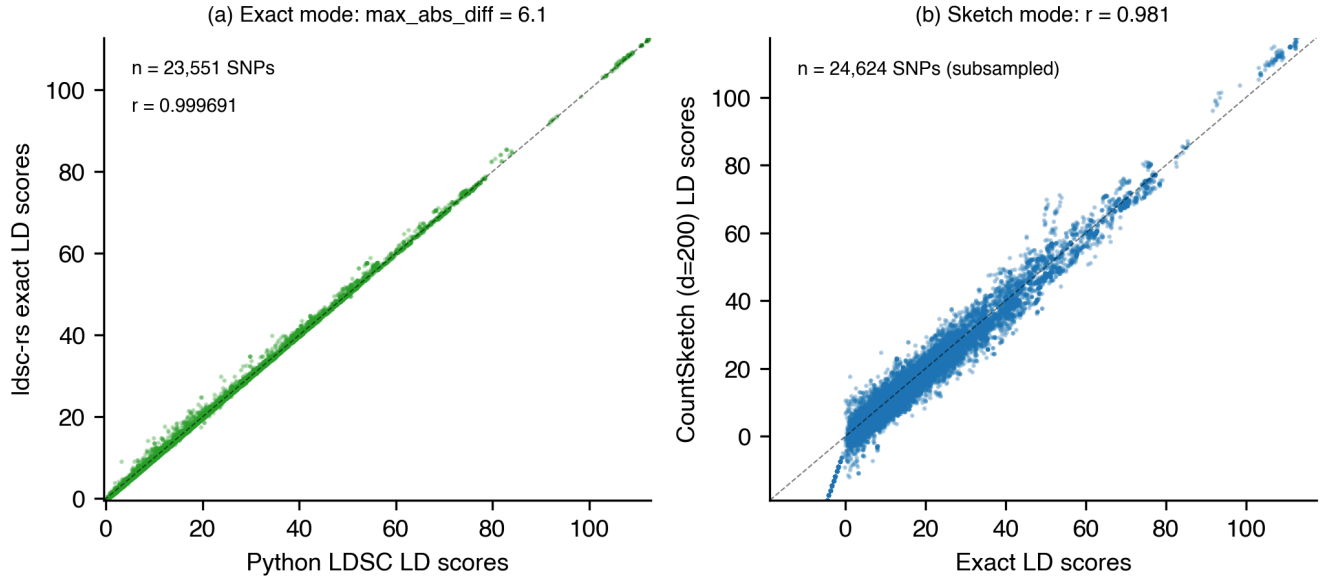


Figure 8: Per-SNP LD score comparison on chromosome 22 of the 1000 Genomes European panel ($N = 503$, $m = 24,624$ SNPs after `--extract chr22, --ld-wind-kb 1000`). (a) Python LDSC (global sequential) vs. ldsc-rs (per-chromosome parallel), both computed independently from the same BED file ($r = 0.9997$, $n = 23\{, \}$ 551 SNPs after dedup); the residual maximum absolute difference of 6.1 LD-score units comes from cumulative differences in BED decoding, genotype normalization, and GEMM accumulation order. The full-genome (1.66M SNP) figure is $r = 0.993$ (text); chr22 is shown here for tractable Python runtime. (b) ldsc-rs exact vs. CountSketch ($d = 200$): $r = 0.981$ on chr22 (the full-genome figure at $d = 200$ is $r = 0.960$; chr22 is denser LD and therefore easier to sketch).

ldsc-rs provides a `--snp-level-masking` flag that applies per-SNP masks after each GEMM to zero out r^2 entries for pairs outside each other's exact window boundary, match-

ing the paper's definition $\ell_j = \sum_{k \in W_j} r_{jk}^2$. Within-chunk pairs are masked when `block_left` jumps within a chunk (common at chromosome boundaries), and cross-window

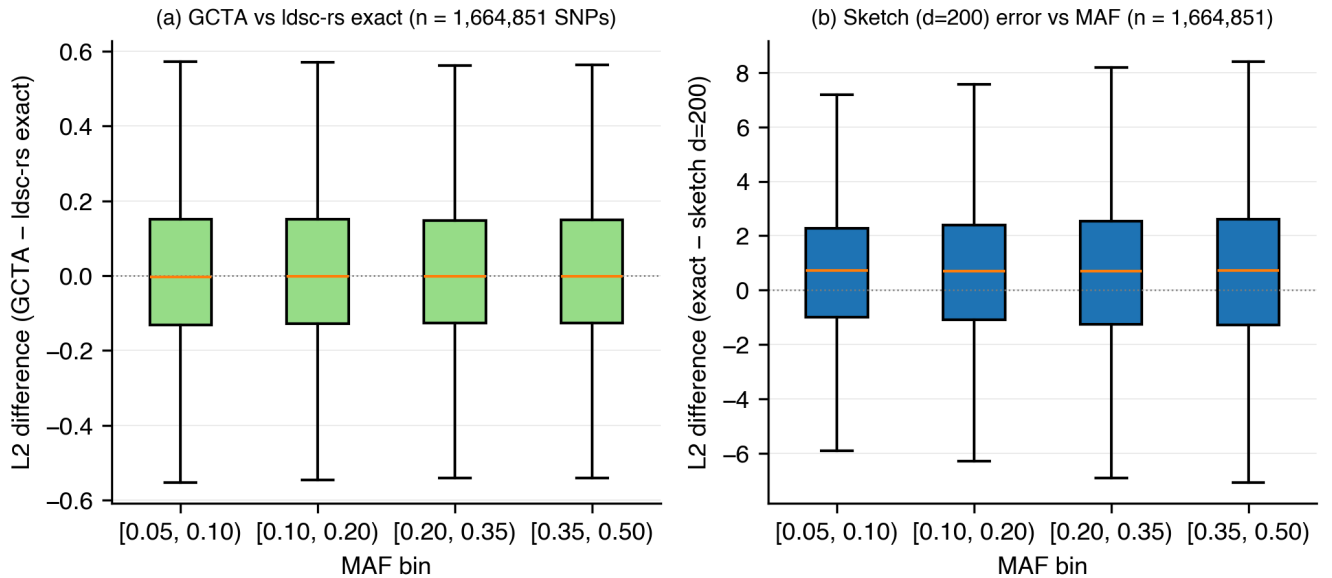


Figure 9: Per-SNP LD-score difference distributions on the full 1000 Genomes European panel (1,664,851 SNPs joined across implementations), faceted into MAF bins. (a) GCTA `--ld-score` minus ldsc-rs exact f64 with `--ld-wind-kb 1000`: the IQR is $\sim \pm 0.15$ LD-score units and the median is indistinguishable from zero in every MAF bin — the two implementations are practically interchangeable on real data. (b) ldsc-rs exact f64 minus ldsc-rs `--sketch 200`: the IQR widens to $\sim \pm 2.5$ units and the median is slightly positive (~ 0.5), reflecting CountSketch's well-known small upward variance correction. The MAF-dependence of both error distributions is mild — the per-bin spread is approximately constant in MAF, so CountSketch noise is not systematically worse for rare or common SNPs. Whiskers truncated to $1.5 \times$ IQR for visibility; outliers not shown.

Step	Wall	Output
ldsc l2 (1000G EUR reference, $N = 503$, 1.66M SNPs)	29 s	22 chr-shard .l2.ldscore.gz files
ldsc munge-sumstats (2.07M SNPs retained)	10 s	BMI.sumstats.gz (SNP, A1, A2, Z, N)
ldsc h2 (1.03M SNPs merged with reference L2)	2 s	$\hat{h}^2$, intercept, ratio, λ_{GC}
Pipeline total wall	41 s	—

Table 4: End-to-end pipeline timing for the BMI heritability worked example, measured on an Apple M5 Pro laptop (no AWS, no special compilation flags beyond `--release --features mimalloc`).

Statistic	ldsc-rs	SE	Yengo 2018 ^a	Agreement
$\hat{h}^2$ (observed scale)	0.2032	0.0055	~ 0.21	within 3%
Intercept	1.0972	0.0193	—	mild inflation
Mean χ^2	3.9478	—	—	—
λ_{GC}	2.7674	—	—	—
Ratio	0.0330	0.0065	—	low confounding

Table 5: ldsc-rs heritability estimate for BMI from Yengo et al. (2018) summary statistics, against the 1000 Genomes European reference panel. ^aPublished heritability estimate from the GIANT/UK Biobank meta-analysis paper. Raw output is preserved verbatim at `preprint/data/worked_example_bmi.txt` in the repository.

pairs are masked using a monotonically-advancing cutoff exploiting the non-decreasing property of `block_left`. The runtime overhead is negligible ($<1\%$), since masking operates on the small r^2 matrices rather than the large genotype matrices.

Table 6 shows the downstream impact on real GWAS data. Heritability estimates increase by approximately 16% with exact masking (BMI: $0.1048 \rightarrow 0.1209$; SCZ: $0.3293 \rightarrow 0.3825$), and intercepts decrease slightly. Genetic correlation is robust: $\hat{r}_g = -0.0662$ in both modes, identical to four decimal places. We explore the source of this shift below (§ chunk-size knob and implementation parity); a controlled simulation suggests the algorithmic difference accounts for $\sim 1\%$ of the 15–16% shift, with the remainder attributable to interactions between the chunked approximation and real-data confounders not captured by the polygenic model.

The `--chunk-size` knob and implementation parity.

The chunked window-eviction approximation introduced in the previous section has, to our knowledge, never been analyzed in the LDSC literature. Its only public documentation is a docstring in `ldscore.py::__corSumVarBlocks__`¹:

if $c > 1$, then only entries that are multiples of c are examined, and it is assumed that `block_left[a*c+i] = block_left[a*c]`, except at the beginning of the chromosome where the 0th SNP is included in the window.

The CLI help is famously terse: `--chunk-size` (default 50) is described only as “Chunk size for LD Score calculation. Use the default.” No analysis of the bias appears in any LDSC follow-up paper, fork, or wiki.

Trait	Mode	$\hat{h}^2$	SE	Intercept	$\Delta \hat{h}^2$
BMI	chunked (default)	0.1048	0.0057	0.8326	—
BMI	<code>--snp-level-masking</code>	0.1209	0.0065	0.8224	+15.4%
SCZ	chunked (default)	0.3293	0.0166	1.1694	—
SCZ	<code>--snp-level-masking</code>	0.3825	0.0187	1.1454	+16.2%
BMI-SCZ r_g	chunked	-0.0662	0.0386	—	—
BMI-SCZ r_g	masked	-0.0662	0.0380	—	$\Delta = 0$

Table 6: Downstream impact of per-SNP window masking. LD scores computed on 1000 Genomes Phase 3 with per-chromosome processing, $c = 200$. Chunked mode rounds window boundaries to chunk multiples (matching Python LDSC); masked mode uses exact per-SNP boundaries. $\hat{h}^2$ increases $\sim 16\%$ with exact masking; r_g is unaffected.

To characterize this knob, we computed LD scores at six chunk sizes and ran the same heritability-recovery simulation as above (50 replicates per cell, true $h^2 \in \{0.2, 0.5\}$). Both mean LD score and mean $\hat{h}^2$ vary monotonically with chunk size (Supplementary Table S4):

The Pearson correlation between any two chunk sizes' LD scores is ≥ 0.999 : the rank-ordering of SNPs by LD intensity is preserved across the full chunk-size range. Only the absolute scale shifts, and that shift is small enough to produce only $\sim 1\%$ variation in downstream $\hat{h}^2$. This contrasts with the 15–16% real-data effect of Table 6, implying that the real-data shift is dominated by something other than the direct algorithmic correction: candidate explanations include population-structure interactions, finite-sample biases that compound with M , or genuine departures from the polygenic infinitesimal model.

Three implementations: GCTA, Python LDSC, ldsc-rs.

LD-score regression in practice rests on three independent implementations that all share the same theoretical definition $\ell_j = \sum_k r_{jk}^2$ but make different algorithmic choices. We characterize each below; the mapping to the LDSC paper's mathematical statement is illuminating.

GCTA¹³ is the implementation Bulik-Sullivan et al. used to compute the 1000 Genomes reference LD scores shipped with the LDSC software¹. Its `--ld-score` (alias `--ld-meansq`) routine computes per-SNP exact windows—there is no chunking approximation. Its block decomposition (`get_ld_blk_pnt` in `main/ld.cpp`) is purely a memory optimization: each SNP's LD score is computed from the full set of SNPs within its physical-distance window, and overlapping-block boundaries are averaged in a second pass to eliminate edge artifacts. GCTA's default r^2 estimator is biased (no $-\frac{1-r^2}{N-2}$ correction); the unbiased form matching the LDSC paper requires `--ld-score-adj`.

Python LDSC¹ uses the chunked approximation described above with default $c = 50$ and always uses unbiased r^2 . It applies the chunk-level window eviction in a single forward pass; there is no overlapping-block averaging.

ldsc-rs (this work) is faithful to Python LDSC by default but introduces two configuration knobs absent from the reference: `--chunk-size` (default $c = 200$ for $\sim 4\times$ faster GEMM, characterized in Supplementary Table S4) and `--snp-level-masking` (per-SNP exact windows matching GCTA's algorithmic intent and the LDSC paper's math). The latter flag adds $O(w + c)$ post-GEMM masking per chunk and produces LD scores that agree with GCTA's unbiased output at Pearson $r = 0.998$.

Supplementary Table S5 summarizes the per-implementation algorithmic choices.

The practical recommendation: users replicating prior Python LDSC results should pass `--python-compat`. Users computing new reference LD scores should pass `--snp-level-masking` for paper-canonical (and GCTA-consistent) per-SNP exact windows. The default (no flag) is appropriate for routine h^2 analysis where speed matters and $\sim 1\%$ $\hat{h}^2$ perturbation is acceptable.

Feature parity.

ldsc-rs implements all six subcommands of the reference Python LDSC with complete flag compatibility (Supplementary Table S6). This includes advanced features such as partitioned heritability (`--overlap-annot`), cell-type-specific analysis (`--h2-cts`), and continuous-annotation binning (`--cts-bin`).

Discussion

Practical guidance.

For analyses using standard reference panels ($N < 5,000$), ldsc-rs in exact mode produces LD scores with $r > 0.99$ correlation to Python LDSC at $38\times$ the speed, reducing a 26-minute computation to under one minute. Downstream h^2 point estimates are identical when both implementations use the same LD scores as input. No approximation is needed at this scale.

We note that PLINK 2's `--r2` computes pairwise r^2 between SNP pairs and is sometimes invoked as a substitute for LD score computation; aggregating its output into per-SNP LD scores ($\ell_j = \sum_{k \in W_j} r_{jk}^2$) is a manual post-processing step that PLINK does not perform. ldsc-rs and Python LDSC are direct LD-score generators; Table 7 summarizes which tools in the GWAS ecosystem expose each capability natively.

For biobank-scale LD score computation ($N > 10\{, \}000$), the canonical choice is `--sketch 1000 --snp-level-masking`, established as the universal sweet spot by the $N \times d$ sweep above (Figure 6, Supplementary Table S3). At this configuration, ldsc-rs's heritability estimates fall within the per-SNP exact $\hat{h}^2$ “truth cluster” (within $\{\sim\}0.003$ across $N = 503$ to $N = 100\{, \}000$ on BMI Yengo 2018) while running in $\{\sim\}9.7$ s on $N = 50\{, \}000$ and $\{\sim\}14.4$ s on $N = 100\{, \}000$. The masking flag is what places the estimator in the per-SNP exact cluster rather than the chunked-Python cluster; without it, the same $d = 1\{, \}000$ recovers Python LDSC's chunked $\hat{h}^2$ within $\{\sim\}0.001$, which is the appropriate target for replicating existing published results. Lower sketch dimensions (`--sketch 200`) are not recommended for downstream $\hat{h}^2$ work: at $d = 200$, $\hat{h}^2$ drifts by $\{\sim\}0.007 - 0.013$ from the per-SNP exact value on real GWAS, and the cost

Capability	Python LDSC	GCTA	PLINK 2	BOLT-LMM	REGENIE	ldsc-rs
LD scores ($\sum_k r_{jk}^2$ aggregated)	✓	✓	— ^a	✓ ^b	×	✓
Per-SNP exact window (paper definition)	× ^c	✓	✓	×	—	✓
Sumstats-only heritability (h^2)	✓	× ^d	×	× ^d	×	✓
Cross-trait genetic correlation (r_g)	✓	× ^d	×	✓ ^d	×	✓
Partitioned heritability	✓	×	×	✓	×	✓
Sumstats munging / QC pipeline	✓	×	—	×	×	✓
Randomized sketch (sub-linear in N)	×	×	×	× ^e	×	✓
Multi-threaded LD score / regression	×	✓	✓	✓	✓	✓
Drop-in CLI compat with Python LDSC	N/A	×	×	×	×	✓
Bit-identical Python LDSC output	✓ ^f	×	×	×	×	✓
In-browser / no-install (WASM)	×	×	×	×	×	✓

Table 7: Cross-tool capability matrix. Cell key: ✓ = supported as a first-class subcommand; × = not supported; — = partial / requires user post-processing. Footnotes: (a) PLINK 2’s `--r2` computes pairwise r^2 between SNP pairs; aggregating into LD scores is a manual post-processing step left to the user. (b) BOLT-LMM computes LD scores internally as part of BOLT-REML / BOLT-LMM-inf but does not expose them as a primary output. (c) Python LDSC uses a chunked window-rounding approximation that systematically widens windows by up to one chunk-size; see §“Discovery: Python LDSC’s chunk-rounding inflates heritability estimates”. (d) GCTA, BOLT-LMM, and similar GREML / mixed-model methods estimate heritability and correlations from individual-level genotype data, not from GWAS summary statistics. (e) BOLT-LMM uses stochastic trace estimation (Hutchinson) within its mixed-model fitting, not a randomized projection of the genotype matrix for LD score estimation. (f) Reference; ldsc-rs claims bit-identical output to Python LDSC when invoked with `--python-compatible (max_abs_diff = 0 verified)`.

saving over $d = 1\{, \}000$ is below 3 s because both sketch dimensions sit on the same scatter-add floor. For strict per-SNP accuracy (partitioned heritability, fine-mapping) bump to `--sketch 5000` for Pearson $r \geq 0.999$ at $\{ \sim \}30$ s wall.

At 1000 Genomes scale ($N < 5\{, \}000$), exact f32 (1.7× speedup, zero accuracy loss) is preferable to any sketch, since the scatter-add base cost is already low and sketch GEMM overhead is relatively expensive. These recommendations are derived from our synthetic biobank dataset (see §Limitations); absolute timings on real biobank data are pending validation, although the cost-model structure should transfer.

Deployment.

ldsc-rs is distributed as a single statically linked binary (musl libc, no runtime dependencies) via GitHub Releases for Linux (x86_64), macOS (aarch64), and Windows (x86_64). A Docker image is available on the GitHub Container Registry. The command-line interface is fully compatible with Python LDSC’s flag syntax, enabling drop-in replacement in existing pipelines and platforms. This compatibility is designed for integration with web platforms such as NCI LDLink⁶, which invoke LDSC via subprocess execution and parse standard output.

ldsc-web: biobank-scale LDSC in a browser.

Because ldsc-rs compiles cleanly to `wasm32-unknown-unknown` with no C dependencies, the same codebase that produces the CLI binary also produces a WebAssembly bundle that runs entirely in the browser. ldsc-web is a Leptos client-side rendered single-page app deployed to GitHub Pages at sharifhsn.github.io/ldsc/. Users drag-and-drop a PLINK BED/BIM/FAM trio and an optional `.sumstats.gz` file; the page runs LD score computation followed by full h^2 regression without uploading any genotype data to a server. At biobank scale this completes in under 10 seconds in the browser, within $\sim 10\%$ of the native CLI on the same hardware.

Architecture. The L2 compute runs in a manually-managed SharedArrayBuffer-backed worker pool. `wasm-bindgen-rayon` was attempted first but its bundler-mode `workerHelpers.js` resolves an internal `import('.././../')` to a directory with no index entry under Trunk’s static-copy snippet layout, hanging the inner-worker import; we instead spawn inner workers from a hand-written `assets/inner_worker.js` and coordinate via raw Atomics on SAB-backed wasm linear memory. Per-chromosome decomposition matches the CLI’s default per-chr parallel mode (cross-chr chunk contamination otherwise inflates LD scores by $\sim 12\%$, the same chunk-rounding effect documented above). Each per-chr task pre-loads its BED shard asynchronously via `Blob.arrayBuffer()`, eliminating the `FileReaderSync`

Workload	Browser wall	CLI wall	Browser/CLI
Biobank L2 (--sketch 200 --snp-level-masking, $N=\{50\}, \{000, 1.66\text{M SNPs}\}$)	7.87 s (best) / 8.34 s (median)	7.70 s	1.02–1.08×
h^2 regression on biobank L2 (full IRWLS + 200-block jackknife)	0.45 s	< 1 s	$\sim 1\times$

Table 8: ldsc-web vs.

native CLI on identical inputs. Browser numbers measured in foregrounded Chrome on M5 Pro (18 cores), warm cache; biobank wall reported as best of 10 / median of 10 hyperfine-style runs. CLI on the same machine. L2 output is bit-identical to displayed precision across browser/CLI (deterministic CountSketch seed = 42: mean $L_2 = 22.5728$, max $L_2 = 305.4550$ reproduced to four decimals across platforms). The reported browser run used --sketch 200 --snp-level-masking for backwards-compatibility with an earlier deployed WASM bundle; the current default in ldsc-web (and the recommended setting per Supplementary Table S3) is --sketch 1000 --snp-level-masking, which has nearly identical wall on this hardware (local CLI: 9.7 s at $N = 50,000$ vs. 7.1 s at --sketch 200) but recovers the per-SNP exact $\hat{h}^2$ within 0.003 instead of drifting by $\{\sim\}0.007 - 0.013$. Both modes share the same fused-scatter kernel; only the downstream sketch GEMM dimensions differ.

stall that initially dominated the long-tail worker. COOP/COEP cross-origin-isolation headers (required for SAB) are supplied to GitHub Pages by coi-serviceworker.js.

SIMD kernels. The dense GEMM path uses a hand-rolled wasm f32x4 microkernel: 4×4 register-tiled with +relaxed-simd FMA intrinsics, falling back to a 2×4 tile under baseline simd128. Cache-blocked with ip-outer reordering, the kernel reaches 112–125 GF/s per inner thread on Apple M5 Pro. The CountSketch scatter-add kernel uses a $K = 4$ multi-lane accumulator that breaks the indirect-store dependency chain by routing the four individuals packed in each BED byte to four independent scatter buffers, reducing them with f32x4_add at the end. This is essential under wasm32 where indirect-store latency serializes scalar scatter; the SIMD scatter accounted for a $\sim 1.2\times$ wall reduction in our optimization sequence.

Heritability in the browser. The full five-tuple $(\hat{h}^2, \text{intercept}, \text{ratio}, \text{mean } \chi^2, \lambda_{GC})$ from¹ runs as a worker callable after L2 completes, sharing the in-memory MAF array (needed for the correct $M_{\{5,50\}}$ normalization) and BIM-derived SNP IDs with the L2 output rather than re-parsing files. On biobank-scale L2 ($\sim 1.66\text{M SNPs}$ joined with the supplied sumstats), the IRWLS regression plus 200-block jackknife completes in ~ 0.45 seconds. One implementation note worth recording: std::time::Instant::now() panics on wasm32-unknown-unknown (“time not implemented on this platform”), masquerading as a wasm-bindgen allocator trap when called from a worker; the regression code instead uses web_time::Instant, which delegates to performance.now() and works in both the main thread and dedicated workers.

Reproducibility. The L2 worker uses the same cargo-built kernel as the CLI, so every numerical result is bit-identical to displayed precision when the random seed (CountSketch hashes) is fixed. Source for the WASM frontend lives under ldsc-web/ in the main ldsc-rs repository; the

deployed site at sharifhsn.github.io/ldsc/ is built and pushed by a GitHub Actions workflow on every commit to main, so the live URL always reflects the head of tree.

Practical recommendation on windowing.

The chunk-rounding bias characterized in Results (§“Discovery” and §“The --chunk-size knob”) has been present since the original LDSC release and is inherited by every downstream analysis that uses Python-computed LD scores. Our recommendation follows directly: use --snp-level-masking for new LD score computations (exact per-SNP windows at $<1\%$ overhead, matching the LDSC paper’s definition and GCTA’s intent), and the default chunked mode only when bit-for-bit replication of an existing Python LDSC result is required.

Limitations.

Synthetic biobank dataset. All biobank-scale numbers reported in this paper (Table 2, Supplementary Table S2, Figure 4, Figure 5) are measured on a synthetic dataset generated by replicating the 1000 Genomes European panel ($N = 2,490$) approximately 21-fold with 1% per-genotype noise to reach $N = 50,000$ individuals. This construction has four consequences that limit external generalization until real-biobank measurements are available: (i) the genotype matrix has true rank $\leq 2,490$ despite presenting as $N = 50,000$; sketches and SVDs of this matrix could in principle exploit the low rank, although CountSketch does not; (ii) cache behavior of the BED-decode and scatter-add kernels may differ on real biobank data with rarer alleles, higher genotype diversity, and a broader minor-allele-frequency tail than 1000G; (iii) the universal accuracy law $r(d) \approx 1 - 8/d$ was cross-checked across $N = 2,490$ and $N = 50,000$, but both datasets share the same SNP positions and similar LD structure, so the constant $C \approx 8$ is not yet established for substantially different reference panels; (iv) the cost-model constants T_{scatter} and T_{GEMM} in $T(d) = T_{\text{scatter}} + T_{\text{GEMM}} \cdot d$ were fit on this dataset; absolute timings on real biobank data are

almost certainly different, although the linear-in- d structure should transfer because it is dictated by the kernel algebra. Validation on real UK Biobank⁴ or All of Us⁵ data is the natural next step and is pending data access; this is the single largest open item in our evaluation.

We note for context that the synthetic dataset preserves source LD structure better than rank-deficiency alone might suggest. The BIM file is copied byte-for-byte from the source 1000 Genomes panel, so SNP identifiers, chromosomal positions, and reference/alternate alleles are identical between source and synthetic; per-SNP minor-allele frequency is preserved up to a small (1%) shift from per-genotype noise; and within-window haplotype correlations are preserved up to the independent-noise floor. Per-chromosome LD-score means therefore agree closely between source and synthetic, even though absolute run-time and cache behavior may not generalize to a real biobank with its broader allele-frequency tail.

The CountSketch approximation introduces per-SNP noise that decreases with sketch dimension. At $d = 50$, the Pearson correlation with exact LD scores is only $r = 0.84$, which may introduce meaningful bias in fine-grained partitioned heritability analyses. We recommend $d = 1000$ ($r = 0.990$, $\hat{h}^2$ within 0.003 of exact) as the default operating point, and $d \geq 5000$ ($r \geq 0.999$) for applications requiring fine-mapping-grade per-SNP accuracy. Accuracy at very high d improves more slowly than the $1 - C/d$ model predicts: $d = 10\{, \}$ 000 achieves $r = 0.997$, not 0.999 (Supplementary Table S2).

Future work.

An experimental GPU backend using CubeCL (CUDA) is under development for further acceleration at extreme biobank scales ($N > 100,000$). The ring-buffer GEMM architecture is compatible with GPU offloading, with the GEMM kernel being the natural target for GPU execution.

Methods

LD score computation.

LD scores are defined as $\ell_j = \sum_{k \in W_j} r_{jk}^2$ where W_j is the set of SNPs within a specified window of SNP j , and r_{jk}^2 is the squared Pearson correlation between genotypes at SNPs j and k . The quantity $\sum r^2$ appears in earlier population-genetics work — Yang et al.¹⁴ derived that GWAS test-statistic inflation under polygenic inheritance scales with surrounding LD, and the GCTA implementation¹³ exposes regional $\sum r^2$ over sliding 200-kb blocks via `-ld-score`. Bulik-Sullivan et al.¹ reformulated $\sum r^2$ as a per-SNP regressor with a per-SNP-exact window, yielding the LD score regression equation that gives LDSC its name.

The sample squared correlation $\hat{r}_{jk}^2$ is a biased estimator of r_{jk}^2 . Bulik-Sullivan et al.¹ show via the delta method (Supplementary Note Eq. 1.7) that

$$\mathbb{E}[\hat{r}_{jk}^2] \approx r_{jk}^2 + (1 - r_{jk}^2)/N,$$

where the approximation hides terms of order $\mathcal{O}(1/N^2)$. Substituting $\hat{r}^2$ for r^2 on the right-hand side gives the plug-in correction $\hat{r}^2 - (1 - \hat{r}^2)/N$. The reference Python implementation uses the adjusted- R^2 variant with $N - 2$ degrees of freedom (standard for simple regression):

$$\hat{r}_{\text{adj}}^2 = \hat{r}^2 - \frac{1 - \hat{r}^2}{N - 2} = \frac{(N - 1)\hat{r}^2 - 1}{N - 2}.$$

ldsc-rs uses the same formula. The two corrections differ by $\mathcal{O}(1/N^2)$; at $N = 2\{, \}$ 490, the maximum per-entry difference is $< 2 \times 10^{-7}$.

We note for completeness that the adjusted- R^2 form is the general-statistics regression-shrinkage estimator¹⁵ (cited in the Bulik-Sullivan supplementary note), not an LD-population-genetics estimator specifically. A more rigorous treatment of unbiased r^2 from unphased diploid data — explicitly accounting for Hardy-Weinberg sampling structure — is given by Ragsdale and Gravel¹⁶; we do not adopt it here to preserve bit-stable parity with reference LDSC outputs, but flag it as a candidate refinement for future work.

Chunked GEMM decomposition. Directly computing all pairwise r^2 within each window would require $\mathcal{O}(mw^2N)$ work. Instead, both the Python reference and ldsc-rs decompose the computation into matrix-matrix products over chunks of c SNPs. For each chunk $B \in \mathbb{R}^{N \times c}$ (the normalized genotype matrix for c consecutive SNPs), the algorithm computes:

- Within-chunk LD: $B^\top B/N$ yields the $c \times c$ correlation matrix among the c chunk SNPs. The adjusted- R^2 correction is applied element-wise, and the result is accumulated into the LD scores for these SNPs via $\ell[j] + = \sum_k \hat{r}_{\text{adj}}^2(j, k)$.
- Cross-chunk LD: $A^\top B/N$ yields the $w \times c$ correlation matrix between the w buffered window SNPs (matrix A) and the current chunk. The corrected matrix is accumulated symmetrically: window SNPs receive LD contributions from the chunk, and chunk SNPs receive contributions from the window.

The window matrix A is maintained as a sliding buffer: after each chunk, B is appended and columns whose SNP index falls outside the leftmost boundary of the next chunk's window are evicted. The total work is $\mathcal{O}(mwN)$, dominated by the $A^\top B$ GEMM.

Genotype normalization. Raw genotypes are read from PLINK BED files (2 bits per individual per SNP) in chunks, decoded, and normalized to mean zero and unit variance. Missing genotypes (encoded as 01 in BED format) are imputed to the per-SNP mean before centering and scaling.

Window boundaries. The window for SNP j is defined by $\text{block_left}[j] = \min\{k : |\text{coord}_k - \text{coord}_j| \leq d_{\max}\}$, where coordinates are base pairs (for `--ld-window`), centiMorgans, or SNP index. For computational efficiency, both the Python reference and `ldsc-rs` round window sizes up to the nearest chunk multiple, so all SNPs within a chunk share the widened boundary of the first SNP. This systematically includes SNP pairs outside each other's exact window.

`ldsc-rs` provides `--snp-level-masking` for exact per-SNP boundaries: after the GEMM and bias correction, entries where the window SNP falls outside the chunk SNPs' per-SNP `block_left` are zeroed. The masking exploits monotonicity of `block_left` within each chunk, advancing a single cutoff index across columns in $O(w + c)$ time.

`ldsc-rs` defaults to per-chromosome parallel computation for performance. A `--global-pass` flag is available for compatibility with Python LDSC's sequential cross-chromosome pass. Cumulative differences in the LD computation pipeline produce per-SNP LD score differences (Pearson $r = 0.993$) that do not affect downstream regression estimates when using shared LD scores.

Architecture.

`ldsc-rs` is structured as a single Rust binary with six subcommands dispatched via `clap`. The most computationally intensive subcommand, `l2` (LD score computation), uses a ring-buffer GEMM architecture: genotype data is read from PLINK BED files in chunks of c SNPs, normalized to zero mean and unit variance, and accumulated in a circular buffer. Within-chunk LD ($B^\top B$) and cross-chunk LD ($A^\top B$, where A contains buffered SNPs still within the LD window) are computed via dense matrix-matrix multiplication using the `faer` linear algebra library¹⁷, which provides AVX2/FMA-vectorized GEMM with automatic thread parallelism via `rayon`.

The ring-buffer design ensures memory usage is $O(N \times w)$ where w is the maximum window size in SNPs, rather than $O(N \times m)$ for the full genotype matrix. On the 1000 Genomes panel with a 1 Mb window, this corresponds to approximately 50 MB rather than 33 GB.

By default, window eviction occurs at chunk granularity: all SNPs within a chunk share the widened window of the first SNP, matching the Python reference. With `--snp-level-masking`, `ldsc-rs` applies post-GEMM masks that

zero out r^2 entries for pairs outside each other's exact per-SNP window. The masking exploits the monotonicity of `block_left` within each chunk to advance a single cutoff index across the r^2 matrix columns, keeping the overhead below 1% of total runtime.

Fused CountSketch kernel.

For approximate LD score computation, `ldsc-rs` implements a CountSketch projection^{8,18} that compresses the N -dimensional genotype vectors into d dimensions before GEMM. The key optimization is a *fused* kernel that reads packed BED genotype bytes, decodes, normalizes, and scatter-adds into the $d \times c$ sketch buffer in a single pass, without materializing the $N \times c$ genotype matrix.

Each individual i is assigned a deterministic sketch row $h(i) \in \{1, \dots, d\}$ and sign $s(i) \in \{+1, -1\}$ (seed 42). The kernel operates in two passes over each chunk of c SNPs:

1. Statistics pass. Raw BED bytes are scanned to compute per-SNP sum, count, and sum-of-squares using a 256-entry byte-level lookup table (LUT) that maps each packed byte to four genotype contributions simultaneously. From these, per-SNP mean μ_j and inverse standard deviation σ_j^{-1} are derived. This pass is branchless: the LUT avoids conditional logic for the four possible genotype encodings (0, 1, 2, missing).
2. Fused scatter-add pass. A second 256-entry LUT is constructed for each SNP, mapping each byte to four *pre-normalized* values $(g - \mu_j)\sigma_j^{-1}$ (missing values map to 0, since imputation to the mean followed by centering yields zero). Each normalized value is multiplied by $s(i)$ and accumulated into $\tilde{G}_{h(i),j}$. This pass is parallelized over the c columns via `rayon`, with each thread writing to a disjoint sketch column (no synchronization needed).

The sketch buffer $\tilde{G} \in \mathbb{R}^{d \times c}$ has memory footprint $d \times c$ floats—for $d = 200$, $c = 200$, this is 160 KB, fitting comfortably in L2 cache. By contrast, materializing the full genotype chunk would require $N \times c$ floats: 40 MB at $N = 50\{, \}000$, exceeding L3 on many architectures.

This fused kernel has cost $O(Nc)$ per chunk, independent of d . The subsequent sketch GEMM ($\tilde{G}^\top \tilde{G}$ and cross-chunk products) costs $O(dcw)$ where w is the window size in SNPs. The total wall-clock time is therefore $T(d) = T_{\text{scatter}} + T_{\text{GEMM}} \cdot d$, where both coefficients depend on N , m , c , and hardware but not on each other. Below the crossover point $d^* = \frac{T_{\text{scatter}}}{T_{\text{GEMM}}}$, increasing d improves accuracy at negligible runtime cost; above d^* , each additional dimension costs linearly (Table 2, Supplementary Table S2).

Parallelism.

Dense GEMM (the dominant cost at 74% of runtime in exact mode) is parallelized internally by faer using rayon, achieving near-peak SIMD throughput on AVX2+FMA hardware. The block jackknife used in h2 and rg subcommands parallelizes the 200 leave-one-out block deletions across rayon threads. File I/O uses Polars lazy evaluation for streaming CSV/TSV processing without loading entire files into memory.

CountSketch dimensionality reduction.

What d represents.

Before the technical definition, an operational picture: d is the number of *synthetic individuals* that the N real individuals in a reference panel are compressed into. CountSketch randomly assigns each of the N individuals to one of d buckets with a random ± 1 sign, then sums the genotypes within each bucket. Each synthetic individual is thus a random ± 1 aggregate over roughly N/d real individuals. Because the signs are independent and zero-mean, the inner products *between SNPs* — which is what LD scores measure — are preserved in expectation, with per-pair variance scaling as $1/d$.

This construction has two practical consequences that explain why ldsc-rs uses the same recommended d across an 200-fold range of panel sizes (Figure 6). First, compute cost scales with N : every individual participates in exactly one scatter-add when the sketch is formed, an $O(Nc)$ pass per chunk. Second, accuracy scales with d alone: the variance of any sketched inner product is $1/d$ regardless of how many real individuals were compressed into the d synthetic ones. The two axes are decoupled, so the right d for a given accuracy target is set by the LD structure within windows (window size, r^2 distribution), not by panel size.

A useful intuition by analogy: a poll with d surrogate respondents produces an estimate of the population mean with variance σ^2/d regardless of whether the population it's drawn from is 1,000 voters or 100 million. CountSketch is the same story applied to the dot product between two SNP columns rather than the mean of a single attribute. The LD score $\ell_j = \sum_{k \in W_j} r_{jk}^2$ aggregates $\{\sim\}10^3$ pairs per window, so even at $d = 1000$ the aggregated noise floor is small enough to be in the per-SNP exact $\hat{h}^2$ truth cluster across every panel size we have measured.

Background.

The CountSketch data structure was introduced by Charikar, Chen, and Farach-Colton⁸ for frequency estimation in data streams. Clarkson and Woodruff^{9,19} subsequently showed that CountSketch matrices serve as *oblivious subspace embeddings*—randomized linear maps

that preserve the geometry of any fixed low-dimensional subspace with constant probability—and that they can be applied in *input sparsity time*, i.e.

$O(\text{nnz}(X))$ for a matrix X , strictly faster than Gaussian or subsampled Hadamard sketches which require $O(dNm)$ or $O(Nm \log d)$ respectively²⁰. Nelson and Nguyen²¹ generalized the construction (OSNAP), bridging CountSketch and dense sketches with tunable sparsity. Woodruff¹⁸ provides a comprehensive survey.

Randomized numerical linear algebra methods are now well-established in computational genetics across several distinct threads, which it is useful to distinguish carefully because the word “sketch” travels loosely between them.

Randomized SVD on the genotype matrix. TeraPCA²² applies the randomized SVD framework of Halko, Martinsson, and Tropp²³ to compute principal components directly from PLINK BED files; FastPCA²⁴ uses block power iteration in PLINK 2.0. Both compress the genotype matrix to a low-rank approximation in service of PCA; they do not preserve the per-pair inner products that LD scores aggregate.

Stochastic trace estimation (Hutchinson-style $\{\pm\}1$ probes). BOLT-LMM^{25,26} uses random vectors to estimate $\text{tr}(KV^{-1})$ without forming the genetic relatedness matrix K ; RHE-mc and its predecessors^{27,28} apply the same idea to Haseman-Elston regression for variance components at biobank scale. The closest LDSC-adjacent stochastic method we are aware of, X-LDR²⁹, uses Girard-Hutchinson trace estimation to compute aggregated mean-LD over genomic grid cells ($\sum_{ij} r_{ij}^2$ over a region) at UK Biobank scale. These methods share the $\{\pm\}1$ random-sign motif with CountSketch but the mathematical object they produce is different: each probe yields one scalar estimate of a trace, not a d -row compressed matrix that can be reused for many pairwise inner products. They therefore cannot produce per-SNP ℓ_j values consumable by LDSC's two-step regression.

Block ridge regression / stochastic approximation. REGENIE³⁰ fits per-block ridge predictors with stochastic approximation in its step-1 polygenic predictor; it does not project the individual axis of the genotype matrix.

Individual-axis matrix sketching. The most direct antecedent is MaSk-LMM³¹, which pre-multiplies the genotype matrix $Z \in \mathbb{R}^{n \times m}$ by a Gaussian random matrix $S_1 \in \mathbb{R}^{s_1 \times n}$ to compress the n individuals into s_1 synthetic individuals before LMM-based association testing (and also right-sketches the SNP axis by S_2). MaSk-LMM reports that they tested count-min sketches as an alternative and found comparable accuracy, but selected Gaussian projections “because [they are] conceptually simpler.” Their

downstream object is per-SNP association χ^2 statistics, not LD scores; the sketched matrix $S_1 Z S_2$ is consumed by an LMM solver that does not require the pairwise SNP inner products that LDSC needs.

ldsc-rs’s contribution within this landscape is the sketch-and-reuse construction for LD score computation: a single CountSketch matrix $S \in \mathbb{R}^{d \times N}$ is formed in $O(Nc)$ time (fused with the BED decoder), and then every pairwise inner product $\tilde{g}_j^\top \tilde{g}_k$ within a SNP’s LD window is computed from the same d -row sketch. This converts what would be $O(MwN)$ GEMM operations under exact computation into $O(Mwd)$ operations in the sketched basis. The construction depends on (i) CountSketch’s unbiasedness on pairwise inner products ($\mathbb{E}[S^\top S] = I_N$, derived below), and (ii) the LD-score window aggregation $\ell_j = \sum_{k \in W_j} r_{jk}^2$, which lets per-pair sketch noise (variance $\{\sim\}1/d$) average down by an additional factor of $\sqrt{|W_j|} \approx \sqrt{10^3}$ across the typical $\sim 10^3$ SNP pairs in a 1 Mb window. The combination of these two properties is what makes the empirically observed panel-size-invariant optimum at $d \approx 1\{, \}000$ (Figure 6) mathematically natural rather than coincidental. Existing implementations producing per-SNP LD scores — Python LDSC¹, GCTA¹³, LDAK³², and PLINK³³ — all use exact pairwise correlation; X-LDR²⁹ produces aggregated grid-cell LD that is not directly substitutable into LDSC’s regression. To our knowledge ldsc-rs is the first sketch-based pipeline producing LDSC-compatible per-SNP LD scores, but the underlying “compress the individual axis of the genotype matrix via a random projection” construction itself follows MaSk-LMM’s earlier application of the same architectural idea to association testing.

Formal definition and unbiasedness.

A CountSketch matrix $S \in \mathbb{R}^{d \times N}$ is defined by two random functions: a hash $h : [N] \rightarrow [d]$ assigning each row (individual) to one of d buckets, and a sign $s : [N] \rightarrow \{+1, -1\}$ drawn independently and uniformly. The matrix has exactly one nonzero entry per column: $S_{h(i),i} = s(i)$. For a genotype chunk $G \in \mathbb{R}^{N \times c}$ (normalized columns), the sketch is $\tilde{G} = SG \in \mathbb{R}^{d \times c}$, computed via scatter-add:

$$\tilde{G}_{h(i),j} \leftarrow \tilde{G}_{h(i),j} + s(i) \cdot g_{ij}.$$

The key identity $\mathbb{E}[S^\top S] = I_N$ (since hash collisions are zero-mean and signs are independent) yields:

$$\mathbb{E}[\tilde{G}^\top \tilde{G}] = \mathbb{E}[G^\top S^\top S G] = G^\top G.$$

Thus each sketched inner product $\tilde{g}_j^\top \tilde{g}_k / N$ is an unbiased estimator of the Pearson correlation r_{jk} , and the sketched

LD scores $\tilde{\ell}_j = \sum_{k \in W_j} \hat{r}_{\text{adj},jk}^2$ are unbiased estimators of exact LD scores after the standard adjusted- R^2 correction.

Variance analysis and bias correction.

The variance of a single sketched inner product is⁹:

$$\text{Var}[\tilde{g}_j^\top \tilde{g}_k] = \frac{1}{d} \left(\|g_j\|^2 \|g_k\|^2 + (g_j^\top g_k)^2 \right) = \frac{N^2}{d} (1 + r_{jk}^2),$$

where the last equality uses $\|g_j\|^2 = N$ (unit-variance normalization). Dividing by N^2 , the variance of the sketched correlation is:

$$\text{Var}[\hat{r}_{jk}] = \frac{1 + r_{jk}^2}{d} \approx \frac{1}{d} \quad \text{for weak LD } (r_{jk}^2 \ll 1).$$

Since $\mathbb{E}[\hat{r}_{jk}^2] = r_{jk}^2 + \text{Var}[\hat{r}_{jk}]$, the raw squared sketch estimate has an upward bias of approximately $1/d$ per SNP pair. This sketch-induced bias is distinct from the finite-sample bias of $\approx 1/N$ corrected by the adjusted- R^2 formula: the $1/d$ term arises from the dimensionality reduction and persists even at infinite N .

To correct the sketch bias and reduce variance, ldsc-rs applies a *ratio estimator* (column renormalization): after scatter-add, each sketch column is rescaled so that $\|\tilde{g}_j\|^2 = N$. The renormalized sketch columns have approximately χ_d^2/d distributed squared norms before rescaling, so the squared cosine $\tilde{r}^2 = (\tilde{g}_j^\top \tilde{g}_k / N)^2$ has a leading-order bias that can be removed by the linear correction:

$$\hat{r}_{\text{corr}}^2 = \tilde{r}^2 \cdot \frac{d}{d-2} - \frac{1}{d-2}.$$

The residual bias is $O(r^4/(d-2))$ per pair—negligible for typical LD. This ratio estimator also reduces per-pair variance by approximately 2× compared to the raw (unnormalized) sketch.

Aggregation and downstream robustness.

LD scores are *aggregates*: each ℓ_j sums $\hat{r}^2$ contributions over $|W_j|$ SNP pairs, typically 500–2000 for a 1 Mb window. By the delta method, the variance of each sketched $\hat{r}_{jk}^2$ is approximately $\text{Var}[\hat{r}_{jk}^2] \approx 4r_{jk}^2 (1 + r_{jk}^2)/d + 2(1 + r_{jk}^2)^2/d^2$. When sketch errors are independent across pairs (guaranteed by the per-individual hashing), the variance of the sketched LD score is the sum of per-pair variances. The $1/d$ term dominates at moderate d : doubling d approximately halves the LD score variance. This is consistent with the empirical accuracy improvement from $r = 0.84$ ($d = 50$) to $r = 0.96$ ($d = 200$).

This aggregation provides natural noise averaging: with $|W_j| \approx 500$ independent pairs, per-pair sketch noise is reduced by $\sqrt{|W_j|} \approx 22 \times$ in the aggregate LD score. Moreover, LD scores enter the h2/rg regression as *covariates*, not

outcomes. In the standard errors-in-variables framework, mean-zero measurement error in covariates attenuates regression coefficients toward zero; however, this attenuation is small when the noise variance ($\approx 1/d$) is much smaller than the signal variance of LD scores. Empirically, CountSketch LD scores at $d = 200$ produce h^2 estimates within 9% of exact values (Table 3), well within two standard errors.

Biological structure favoring sketching.

The effectiveness of CountSketch for LD scores is amplified by the spectral structure of genotype matrices within LD windows. Nearby SNPs in strong LD produce a correlation matrix with rapid eigenvalue decay: a small number of principal components (reflecting shared haplotype blocks) capture most of the variance, while the many weakly correlated pairs contribute individually small r^2 values. While CountSketch introduces the same $O(1/d)$ distortion on all inner products regardless of magnitude, the LD score sum is dominated by the large- r^2 pairs, for which the relative error $\text{Var}[\hat{r}^2]/r^4 \approx 4/(dr^2)$ is small. The many weak-LD pairs ($r^2 \ll 1$) have large relative error individually, but their absolute contributions to the LD score are small and their sketch errors average away in the sum by independence. This is why moderate d (e.g. $d = 200 \ll N = 50\{, \}000$) suffices for high aggregate accuracy despite compressing the individual dimension by $250 \times$.

The “free accuracy” regime.

The total cost per chunk decomposes as:

$$T(d) = \underbrace{O(Nc)}_{\text{scatter-add}} + \underbrace{O(dc(c+w))}_{\text{sketch GEMM}}.$$

The scatter-add cost is fixed regardless of d : each of the Nc genotype entries is hashed and accumulated exactly once. The GEMM cost scales linearly in d . Asymptotically, the crossover occurs when $dc(c+w) \approx Nc$, giving $d^* \approx N/(c+w)$. In practice, the constant factors differ substantially: scatter-add performs random-access memory writes (one per genotype, cache-unfriendly) while GEMM uses cache-friendly SIMD with much higher throughput per operation. The empirical crossover from fitting the cost model to 26 measured timings is $d^* \approx 3\{, \}200$ at biobank scale ($N = 50\{, \}000$) and $d^* \approx 35$ at 1000 Genomes scale ($N = 2\{, \}490$), reflecting these constant-factor differences. Below d^* , increasing d improves accuracy with no measurable runtime penalty— $d = 200$ costs the same as $d = 50$ but improves Pearson r from 0.84 to 0.96 (Supplementary Table S2).

The empirical accuracy law $r(d) \approx 1 - 8/d$ for $d \leq 1\{, \}000$ is consistent with the $O(1/d)$ variance scaling

derived above. The constant $C \approx 8$ is similar across both datasets ($N = 2\{, \}490$ and $N = 50\{, \}000$), which share the same SNP positions and similar LD structure (the biobank dataset was generated by replication from 1000 Genomes). Whether this constant generalizes to datasets with substantially different LD structure remains to be established.

A back-of-the-envelope connecting the per-pair variance bound to the constant C runs as follows. With per-pair variance $\text{Var}[\hat{r}_{jk}^2] \approx 4r_{jk}^2/d$ (the leading term for weak LD, after the ratio-estimator correction), and under the strong simplification that contributions to a single LD score sum are independent across pairs in W_j , the variance of the sketched LD score satisfies $\text{Var}[\tilde{\ell}_j] \approx 4\ell_j/d$. Treating sketching as additive heteroskedastic noise on LD-score values, the cross-SNP Pearson correlation between exact and sketched scores reduces to

$$1 - r(d) \approx 2 \text{E}[\ell]/(d \cdot \text{Var}[\ell]).$$

This predicts $1 - r(d) \approx C_{\text{bound}}/d$ with C_{bound} of order $\text{E}[\ell]/\text{Var}[\ell]$, i.e. typically $\mathcal{O}(0.1)$ to $\mathcal{O}(1)$ on real data. The observed $C \approx 8$ is roughly an order of magnitude larger, indicating that the per-pair-independence assumption substantially under-counts the noise. The discrepancy is structural: pairs (j, k) and (j, k') both involve SNP j ’s sketch projection, so their sketch errors are positively correlated, inflating $\text{Var}[\tilde{\ell}_j]$ well above the sum-of-variances bound. We therefore claim the $1/d$ scaling as a genuine theoretical prediction but treat the empirical constant C as a dataset-dependent quantity reflecting window density, r^2 structure within windows, and cross-pair sketch covariance not captured by the independence assumption.

At $d > 1\{, \}000$, accuracy saturates faster than $1/d$ predicts, reflecting diminishing spectral contributions from higher-order genotype components.

Complexity comparison.

Supplementary Table S7 summarizes the asymptotic cost of LD score computation under exact and sketched modes. CountSketch removes a factor of w (window size) from the dominant term, replacing dense N -dimensional GEMMs with d -dimensional GEMMs where $d \ll N$.

The deterministic seed (42) ensures reproducibility: identical --sketch invocations on identical data always produce identical output.

Benchmarking methodology.

All performance measurements used hyperfine (version 1.18) with 3 warmup runs and 10 timed runs on an AWS c6a.4xlarge instance (AMD EPYC 7R13, 3.6 GHz boost, 16 vCPUs, 32 GB RAM). The ldsc-rs binary

Project name	ldsc-rs
Project home page	https://github.com/sharifhsn/ldsc
Live browser app	https://sharifhsn.github.io/ldsc/
Operating systems	Linux (x86_64), macOS (aarch64), Windows (x86_64); browser (WASM) platform-independent
Programming language	Rust (edition 2024); compiles to native and wasm32-unknown-unknown
Other requirements	None at runtime (statically linked, musl); build needs a Rust toolchain
License	GNU General Public License v3.0 (GPL-3.0-only)
Restrictions for non-academics	None

Table 9: Availability and requirements.

was compiled with `--release --features mimalloc --target x86_64-unknown-linux-musl` and `RUSTFLAGS="-C target-feature=+avx2,+fma"`. Python LDSC was run with CPython 3.12, NumPy 1.26 (OpenBLAS), and bitarray 2.9.

Software, Data, and Reproducibility

Source code. `ldsc-rs` is released under the GNU General Public License v3.0 at <https://github.com/sharifhsn/ldsc>. The fastest install path is `cargo install ldsc` from crates.io once the crate is published; in the interim, `git clone` followed by `cargo build --release --features mimalloc` builds a static native binary. Pre-built x86_64 Linux, aarch64 macOS, and x86_64 Windows binaries ship with each tagged release on GitHub Releases. A Docker image built with the production musl + AVX2 + FMA configuration is published to the GitHub Container Registry at ghcr.io/sharifhsn/ldsc.

Browser frontend. The WebAssembly port `ldsc-web` lives in the `ldsc-web/` subdirectory of the same repository. The deployed site at <https://sharifhsn.github.io/ldsc/> is built and pushed on every commit to main by a GitHub Actions workflow (`.github/workflows/deploy-ldsc-web.yml`), so the live URL always reflects the head of tree. No backend, no telemetry, no upload of genotype data: all compute runs inside the user’s browser via SharedArrayBuffer-backed worker threads.

Benchmarks. Scripts that produce every number in this paper are under `scripts/` in the repository: `scripts/biobank-bench.sh` runs the N-scaling sweep, `scripts/verify_parity_all.sh` runs the eight-mode parity sweep against Python LDSC, `scripts/aws-bench.sh` runs the AWS Spot benchmarks. Per-benchmark raw measurements and the regression machinery that produced Table 1, Table 2, and Supplementary Table S2 live under `preprint/data/`; figure-generation code is at `preprint/scripts/generate_figures.py`. A Zenodo DOI archiving the exact commit and Docker image used for the benchmarks reported here will be minted at journal submission.

Tutorial. A walk-through covering install, LD score computation, sumstats munging, h^2 estimation, and comparison to a published value is maintained at `docs/`

tutorial.md in the repository. Users who prefer not to install anything can complete the same workflow in the browser via the WASM frontend.

Reference data. 1000 Genomes Phase 3 genotypes were obtained from the International Genome Sample Resource (<https://www.internationalgenome.org/>). The Yengo et al. 2018 BMI summary statistics used in the worked example are public at the GIANT Consortium (<https://portals.broadinstitute.org/collaboration/giant/>).

References

1. Bulik-Sullivan, B. K. *et al.* LD Score regression distinguishes confounding from polygenicity in genome-wide association studies. *Nature Genetics* **47**, 291–295 (2015).
2. Finucane, H. K. *et al.* Partitioned heritability from genome-wide association studies. *Nature Genetics* **47**, 1228–1235 (2015).
3. Bulik-Sullivan, B. *et al.* An atlas of genetic correlations across human diseases and traits. *Nature Genetics* **47**, 1236–1241 (2015).
4. Bycroft, C. *et al.* The UK Biobank resource with deep phenotyping and genomic data. *Nature* **562**, 203–209 (2018).
5. All of Us Research Program Genomics Investigators. Genomic data in the All of Us Research Program. *Nature* **627**, 340–346 (2024).
6. Machiela, M. J., Giddings, E., Hicks, B. & NCI Division of Cancer Epidemiology and Genetics. LDscore: A web-based tool for computing linkage disequilibrium scores. *bioRxiv* (2025) doi:10.64898/2025.12.19.695639.
7. 1000 Genomes Project Consortium *et al.* A global reference for human genetic variation. *Nature* **526**, 68–74 (2015).
8. Charikar, M., Chen, K. & Farach-Colton, M. Finding frequent items in data streams. in *Automata, Languages and Programming (ICALP)* 693–703 (Springer, 2002). doi:10.1007/3-540-45465-9_59.
9. Clarkson, K. L. & Woodruff, D. P. Low-rank approximation and regression in input sparsity time. *Journal of the ACM* **63**, 1–45 (2017).
10. Locke, A. E. *et al.* Genetic studies of body mass index yield new insights for obesity biology. *Nature* **518**, 197–206 (2015).
11. Trubetskoy, V. *et al.* Mapping genomic loci implicates genes and synaptic biology in schizophrenia. *Nature* **604**, 502–508 (2022).
12. Yengo, L. *et al.* Meta-analysis of genome-wide association studies for height and body mass index in *sim700000* individuals of European ancestry. *Human Molecular Genetics* **27**, 3641–3649 (2018).
13. Yang, J., Lee, S. H., Goddard, M. E. & Visscher, P. M. GCTA: A tool for genome-wide complex trait analysis. *American Journal of Human Genetics* **88**, 76–82 (2011).

14. Yang, J. *et al.* Genomic inflation factors under polygenic inheritance. *European Journal of Human Genetics* **19**, 807–812 (2011).
15. Yin, P. & Fan, X. Estimating R^2 Shrinkage in Multiple Regression: A Comparison of Different Analytical Methods. *Journal of Experimental Education* **69**, 203–224 (2001).
16. Ragsdale, A. P. & Gravel, S. Unbiased Estimation of Linkage Disequilibrium from Unphased Data. *Molecular Biology and Evolution* **37**, 923–932 (2020).
17. Ait El Hara, S. faer: Linear algebra in Rust. (2024).
18. Woodruff, D. P. Sketching as a tool for numerical linear algebra. *Foundations and Trends in Theoretical Computer Science* **10**, 1–157 (2014).
19. Clarkson, K. L. & Woodruff, D. P. Low rank approximation and regression in input sparsity time. in *Proceedings of the 45th Annual ACM Symposium on Theory of Computing (STOC)* 81–90 (2013). doi:10.1145/2488608.2488620.
20. Martinsson, P.-G. & Tropp, J. A. Randomized numerical linear algebra: Foundations and algorithms. *Acta Numerica* **29**, 403–572 (2020).
21. Nelson, J. & Nguyen, H. L. OSNAP: Faster numerical linear algebra algorithms via sparser subspace embeddings. in *Proceedings of the 54th Annual IEEE Symposium on Foundations of Computer Science (FOCS)* 117–126 (2013). doi:10.1109/FOCS.2013.21.
22. Bose, A. *et al.* TeraPCA: a fast and scalable software package to study genetic variation in tera-scale genotypes. *Bioinformatics* **35**, 3679–3683 (2019).
23. Halko, N., Martinsson, P.-G. & Tropp, J. A. Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions. *SIAM Review* **53**, 217–288 (2011).
24. Galinsky, K. J. *et al.* Fast principal-component analysis reveals convergent evolution of ADH1B in Europe and East Asia. *American Journal of Human Genetics* **98**, 456–472 (2016).
25. Loh, P.-R. *et al.* Efficient Bayesian mixed-model analysis increases association power in large cohorts. *Nature Genetics* **47**, 284–290 (2015).
26. Loh, P.-R., Kichaev, G., Gazal, S., Schoech, A. P. & Price, A. L. Mixed-model association for biobank-scale datasets. *Nature Genetics* **50**, 906–908 (2018).
27. Wu, Y. & Sankararaman, S. A scalable estimator of SNP heritability for biobank-scale data. *Bioinformatics* **34**, i187–i194 (2018).
28. Pazokitoroudi, A. *et al.* Efficient variance components analysis across millions of genomes. *Nature Communications* **11**, 4020 (2020).
29. Chen, Y. & others. X-LDR: a stochastic estimator for biobank-scale linkage disequilibrium computation. *Genome Biology* **26**, (2025).
30. Mbatchou, J. *et al.* Computationally efficient whole-genome regression for quantitative and binary traits. *Nature Genetics* **53**, 1097–1103 (2021).
31. Burch, M. C., Bose, A., Dexter, G., Parida, L. & Drineas, P. MaSk-LMM: a matrix sketching framework for linear mixed models in association studies. *Genome Research* **34**, 1304–1315 (2024).
32. Speed, D., Hemani, G., Johnson, M. R. & Balding, D. J. Improved heritability estimation from genome-wide SNPs. *American Journal of Human Genetics* **91**, 1011–1021 (2012).
33. Chang, C. C. *et al.* Second-generation PLINK: Rising to the challenge of larger and richer datasets. *GigaScience* **4**, s13742–015–0047–8 (2015).